```
// G03AdPoints.cs
namespace Sy.G03.Boot
{
    /// &lt;summary&gt;
    /// G03 rewarded point ids (task S2-T002). Namespaced with the g03_ prefix so no
    /// G03 call can ever draw P02/P06/G02 category budget and vice versa. Category
    /// mapping lives in Sy.Platform.AdCategories.CategoryOfPoint: RescuePurge = the
    /// waste-heat valve purge offer (rescue 4/day), HybridLuck = the cross-type
    /// hybrid probability doubling offer (double 5/day), DexPick = the dex album
    /// direct pick offer (direct 1/day), all under the 12/day global cap.
    /// &lt;/summary&gt;
    public static class G03AdPoints
    {
        public const string RescuePurge = "g03_rescue_purge";
        public const string HybridLuck = "g03_hybrid_luck";
        public const string DexPick = "g03_dex_pick";
    }
}
// G03BootServices.cs
using Sy.Core;
using Sy.Platform;
using Sy.Telemetry;
using Logger = Sy.Core.Logger;
namespace Sy.G03.Boot
{
    /// &lt;summary&gt;
    /// Platform suite wiring for G03 (task S2-T002, G02 BootServices paradigm).
    /// PlatformBootstrap.Create(new AdCapPolicy()) resolves editor/SY_MOCK to the
    /// mock suite and real WeChat builds to the Wx suite, so the editor path and a
    /// production build differ only by define, never by wiring. Future offer faces
    /// consume from the SAME registered ads service the pre-check reads
    /// (HasCategoryBudget), keeping pre-check and consumption consistent (P02
    /// T006-B2 pattern). Zero-server red line 12: the TCB batch sink class stays
    /// available in the shared package but is NEVER registered here.
    /// &lt;/summary&gt;
    public static class G03BootServices
    {
        public static void RegisterAll()
        {
            if (ServiceLocator.TryGet&lt;IAdsService&gt;(out _)) return; // idempotent: double register would throw
            var suite = PlatformBootstrap.Create(new AdCapPolicy());
            ServiceLocator.Register&lt;IAdsService&gt;(suite.Ads);
            ServiceLocator.Register&lt;IPlatformService&gt;(suite.Platform);
            ServiceLocator.Register&lt;IStorage&gt;(suite.Storage);
            // S4-T004: the launch-entry query face joins the locator so the boot
            // host can read the share attribution exactly once (ILaunchInfo contract).
            ServiceLocator.Register&lt;ILaunchInfo&gt;(suite.Launch);
#if SY_WECHAT &amp;&amp; !UNITY_EDITOR
            // Wx custom analytics joins the telemetry pipeline via
            // G03TelemetryWire.CreateService; mock/editor builds compile this out
```

```
            // entirely and the debug sink stays the only one.
            ServiceLocator.Register&lt;IAnalyticsReporter&gt;(new G03WxAnalyticsReporter());
#endif
            Logger.Info("BOOT", "g03 platform suite registered (suite=" + PlatformBootstrap.SuiteIdFor(suite.Channel)
                + ", g03 ad categories: rescue 4 / double 5 / direct 1, global 12/day)");
        }
    }
}
// G03TelemetryWire.cs
using Sy.Core;
using Sy.Telemetry;
namespace Sy.G03.Boot
{
    /// &lt;summary&gt;
    /// Telemetry assembly point for G03 (task S2-T002, G02TelemetryWire paradigm):
    /// DebugLogSink is always attached; the Wx analytics sink joins when a reporter
    /// is available (explicit param wins, otherwise the locator is consulted for
    /// the platform-registered Wx reporter on real builds). Zero-server red line 12:
    /// the TCB batch sink class stays available in the shared package but is never
    /// registered. Event schema faces (hook funnel, ad offer funnel) land with the
    /// slices that own those events.
    /// &lt;/summary&gt;
    public static class G03TelemetryWire
    {
        public static TelemetryService CreateService(IAnalyticsReporter reporter = null)
        {
            var telemetry = new TelemetryService();
            telemetry.AddSink(new DebugLogSink());
            if (reporter == null &amp;&amp; ServiceLocator.TryGet&lt;IAnalyticsReporter&gt;(out var fromLocator))
                reporter = fromLocator;
            if (reporter != null) telemetry.AddSink(new WxAnalyticsSink(reporter));
            return telemetry;
        }
    }
}
// G03WxAnalyticsReporter.cs
#if SY_WECHAT
using System.Runtime.InteropServices;
using Sy.Telemetry;
namespace Sy.G03.Boot
{
    /// &lt;summary&gt;
    /// WeChat analytics reporter (task S2-T002, G02 paradigm): bridges the telemetry
    /// WxAnalyticsSink onto the platform JS entry SyReportAnalytics declared in
    /// SyWxBridge.jslib. Compiled only for real WeChat builds (no UNITY_EDITOR);
    /// mock/editor builds compile this out and the debug sink stays the only one.
    /// The duplicate extern declaration binds to the same jslib function the
    /// Sy.Platform Wx bridge uses.
    /// &lt;/summary&gt;
    public sealed class G03WxAnalyticsReporter : IAnalyticsReporter
```

```
    {
        [DllImport("__Internal", EntryPoint = "SyReportAnalytics")]
        static extern void ReportBridge(string evtId, string fieldsJson);
        public void Report(string evtId, string fieldsJson)
            =&gt; ReportBridge(evtId ?? "", fieldsJson ?? "{}");
    }
}
#endif
// G03AudioCatalog.cs
// G03AudioCatalog.cs - the per-project audio catalog (the G09 HospitalAudio
// X027 paradigm applied to the merge-mania clinic of G03): a ScriptableObject
// that serializes clip references by filename stem, plus the null-safe
// runtime lookup. The asset is built by the editor tool (AudioSetup) from
// Assets/_Game/Audio; runtime consumers MUST treat a missing catalog/clip
// as "skip silently" so headless checks stay green on any project state.
// Delivered stems so far: g03_bgm_main (the manifest BGM row) and
// g03_sting_share (the S4 share sting, consumption seat follows the import).
using System;
using System.Collections.Generic;
using UnityEngine;
namespace Sy.G03
{
    /// &lt;summary&gt;
    /// Generated-audio catalog (X027 paradigm): the AudioSetup editor
    /// tool scans Assets/_Game/Audio and serializes clip references
    /// here so runtime code resolves audio by filename stem. The asset
    /// lives at Assets/_Game/Resources/AudioCatalog.asset (Resources
    /// keeps runtime lookup build-safe with zero Addressables wiring).
    /// &lt;/summary&gt;
    [CreateAssetMenu(fileName = "AudioCatalog", menuName = "Sy/G03 Audio Catalog")]
    public sealed class AudioCatalog : ScriptableObject
    {
        [Serializable]
        public sealed class Entry
        {
            public string key;      // filename stem, e.g. "g03_bgm_main"
            public AudioClip clip;  // direct reference keeps builds self-contained
        }
        public List&lt;Entry&gt; entries = new List&lt;Entry&gt;();
        Dictionary&lt;string, AudioClip&gt; _index;
        void OnEnable()
        {
            // ScriptableObject instances can be revived with a stale cache;
            // drop it so the next Get() rebuilds from entries.
            _index = null;
        }
        public AudioClip Get(string key)
        {
            if (string.IsNullOrEmpty(key)) return null;
            if (_index == null)
```

```
            {
                _index = new Dictionary&lt;string, AudioClip&gt;(entries.Count, StringComparer.Ordinal);
                for (int i = 0; i &lt; entries.Count; i++)
                {
                    var e = entries[i];
                    if (e != null &amp;&amp; !string.IsNullOrEmpty(e.key) &amp;&amp; e.clip != null &amp;&amp; !_index.C
                        _index[e.key] = e.clip;
                }
            }
            return _index.TryGetValue(key, out var c) ? c : null;
        }
    }
    /// &lt;summary&gt;
    /// Runtime lookup helper. Every consumer MUST treat null as "clip
    /// missing -&gt; skip" so headless checks stay green on any project
    /// state (catalog absent, audio pruned, name typos).
    /// &lt;/summary&gt;
    public static class AudioRuntime
    {
        static AudioCatalog _catalog;
        public static AudioCatalog Catalog
        {
            get
            {
                if (_catalog == null) _catalog = Resources.Load&lt;AudioCatalog&gt;("AudioCatalog");
                return _catalog;
            }
        }
        public static AudioClip Get(string key)
        {
            var c = Catalog;
            return c != null ? c.Get(key) : null;
        }
    }
}
// G03DexGate.cs
namespace Sy.G03
{
    /// &lt;summary&gt;
    /// Dex collection milestone gate (task S2-T004, G02 ChapterGate paradigm): converts
    /// the discovered-dex count into hall chapter unlocks. Pure deterministic math so
    /// EditMode tests pin every threshold; the dex hall and the opening ceremony read
    /// the same ladder. Chapter ids are 1-based: 1 = the museum opens with the first
    /// hatch, 2/3/4 = collection wings at 4/8/12 of the 12 hidden species.
    /// &lt;/summary&gt;
    public static class G03DexGate
    {
        /// &lt;summary&gt;Dex counts at which a new hall chapter unlocks (ascending, 1-based).&lt;/summary&gt;
        public static readonly int[] ChapterThresholds = { 1, 4, 8, 12 };
        /// &lt;summary&gt;Highest chapter id reachable (matches the catalog's 12 dex rows).&lt;/summary&gt;
```

```
        public const int MaxChapter = 4;
        /// &lt;summary&gt;Chapters unlocked by the collected count: how many thresholds are crossed.&lt;/summary&gt;
        public static int ChapterCountFor(int dexCollected)
        {
            int count = 0;
            for (int i = 0; i &lt; ChapterThresholds.Length; i++)
            {
                if (dexCollected &gt;= ChapterThresholds[i]) count++;
            }
            return count;
        }
        /// &lt;summary&gt;
        /// Chapter ids crossed beyond the previous count, ascending (the G02 NewlyUnlocked
        /// shape). An empty result means "no new chapter" - callers must treat that as a
        /// quiet pass, never as an error.
        /// &lt;/summary&gt;
        public static int[] NewlyUnlocked(int previousChapterCount, int dexCollected)
        {
            int current = ChapterCountFor(dexCollected);
            if (previousChapterCount &lt; 0) previousChapterCount = 0;
            if (current &lt;= previousChapterCount) return new int[0];
            var ids = new int[current - previousChapterCount];
            for (int i = previousChapterCount; i &lt; current; i++) ids[i - previousChapterCount] = i + 1;
            return ids;
        }
    }
}
// G03GameCatalog.cs
using System;
using System.Collections.Generic;
using UnityEngine;
using Logger = Sy.Core.Logger;
namespace Sy.G03
{
    [Serializable]
    public sealed class TierItemData
    {
        public int species;      // MergeSpecies: 0 produce / 1 creature
        public int tier;         // 0..TopTier (5)
        public string id;        // p0..p5 / c0..c5
        public string nameKey;    // i18n key for the display name
        public string colorHex;   // jelly-block tint for the procedural placeholder
    }
    [Serializable]
    public sealed class DexEntryData
    {
        public string id;           // h01..h12
        public int produceTier;    // produce line tier of the parent pair
        public int creatureTier;    // creature line tier of the parent pair
        public float chance;        // hybrid probability on a qualifying cross-type contact
```

```
    public string nameKey;      // i18n key for the hidden species name
    public string quipKey;      // i18n key for the roast line (first-laugh signal, red line 11)
    public string colorHex;     // dex card tint
}
[Serializable]
public sealed class SecondDexEntryData
{
    public string id;          // s01..s12 (second-lineage slots, W1-b)
    public int baseSpecies;    // fusion partner base line (MergeSpecies: 0 produce / 1 creature)
    public int baseTier;       // 0..TopTier (selection key = the base item's identity)
    public string nameKey;     // i18n key for the second-lineage name (W1-c album page consumes)
    public string quipKey;     // i18n key for the roast line (W1-c album page consumes)
}
[Serializable]
public sealed class CatalogEconomyData
{
    public int spawnMinTier;    // mid-board spawn window lower bound (E4 decision: 1..TopTier)
    public int spawnMaxTier;    // upper bound, anchored to graybox TopTier
}
[Serializable]
public sealed class CatalogFile
{
    public TierItemData[] tierItems;
    public DexEntryData[] dex;
    public SecondDexEntryData[] secondDex;
    public CatalogEconomyData economy;
}
/// &lt;summary&gt;
/// Static game catalog for MergeMania (task S1-T002). Data layer only: the dex
/// table carries the 12 hidden species plus their roast-line keys, the lineage
/// face of each dex row is the (produceTier, creatureTier) parent pair, and the
/// tier item table feeds the S1-T003 board rendering. Graybox parity anchor:
/// the TopTier x TopTier pair maps to h12 (cat-melon) with chance 1.0 - the
/// ported MergeSim resolves exactly that pair deterministically, so the data
/// table must never disagree with it.
/// &lt;/summary&gt;
public sealed class G03GameCatalog
{
    public TierItemData[] TierItems { get; }
    public DexEntryData[] Dex { get; }
    public SecondDexEntryData[] SecondDex { get; }
    public CatalogEconomyData Economy { get; }
    G03GameCatalog(CatalogFile file)
    {
        TierItems = file.tierItems;
        Dex = file.dex;
        SecondDex = file.secondDex;
        Economy = file.economy;
    }
    public static G03GameCatalog Load()
```

```
{
    var asset = Resources.Load&lt;TextAsset&gt;("cfg_g03_catalog");
    if (asset == null)
    {
        Logger.Warn("G03.Catalog", "cfg_g03_catalog.json not found under Resources");
        return null;
    }
    var file = JsonUtility.FromJson&lt;CatalogFile&gt;(asset.text);
    if (file == null || file.tierItems == null || file.tierItems.Length == 0
        || file.dex == null || file.dex.Length == 0
        || file.secondDex == null || file.secondDex.Length == 0)
    {
        Logger.Warn("G03.Catalog", "cfg_g03_catalog.json failed to parse");
        return null;
    }
    return new G03GameCatalog(file);
}
public TierItemData TierItem(int species, int tier)
{
    foreach (var t in TierItems)
    {
        if (t.species == species &amp;&amp; t.tier == tier) return t;
    }
    return null;
}
public DexEntryData DexById(string id)
{
    foreach (var d in Dex)
    {
        if (d.id == id) return d;
    }
    return null;
}
/// &lt;summary&gt;
/// Lineage lookup: which hidden species a (produceTier, creatureTier) parent
/// pair can hatch. Tier-0 pairs are out of scope (the spawn window starts at
/// tier 1), and unknown pairs return null - callers must treat that as
/// "no hybrid", never as an error.
/// &lt;/summary&gt;
public DexEntryData HybridRule(int produceTier, int creatureTier)
{
    foreach (var d in Dex)
    {
        if (d.produceTier == produceTier &amp;&amp; d.creatureTier == creatureTier) return d;
    }
    return null;
}
/// &lt;summary&gt;
/// Second-lineage slot lookup (U153-D04 W1-b): which s01-s12 slot a
/// hybrid x base fusion lights, keyed by the base item's (species, tier).
```

```
/// Unknown keys return null - callers treat that as "no slot", never
/// as an error.
/// &lt;/summary&gt;
public SecondDexEntryData SecondRuleOf(int baseSpecies, int baseTier)
{
    foreach (var d in SecondDex)
    {
        if (d.baseSpecies == baseSpecies &amp;&amp; d.baseTier == baseTier) return d;
    }
    return null;
}
/// &lt;summary&gt;
/// Structural validation shared by the EditMode tests and any future gate.
/// Returns the list of problems; empty means the table is well-formed.
/// &lt;/summary&gt;
public List&lt;string&gt; Validate()
{
    var problems = new List&lt;string&gt;();
    var ids = new HashSet&lt;string&gt;();
    foreach (var t in TierItems)
    {
        if (string.IsNullOrEmpty(t.id)) problems.Add("tierItem missing id");
        if (!ids.Add("item:" + t.id)) problems.Add("duplicate tierItem id " + t.id);
        if (t.species != 0 &amp;&amp; t.species != 1)
            problems.Add("tierItem " + t.id + " species must be 0/1, got " + t.species);
        if (t.tier &lt; 0 || t.tier &gt; MergeSim.TopTier)
            problems.Add("tierItem " + t.id + " tier out of 0..TopTier");
        if (!ColorHexParses(t.colorHex))
            problems.Add("tierItem " + t.id + " colorHex does not parse");
    }
    var dexIds = new HashSet&lt;string&gt;();
    var pairs = new HashSet&lt;string&gt;();
    foreach (var d in Dex)
    {
        if (string.IsNullOrEmpty(d.id)) problems.Add("dex entry missing id");
        if (!dexIds.Add(d.id)) problems.Add("duplicate dex id " + d.id);
        string pair = d.produceTier + "x" + d.creatureTier;
        if (!pairs.Add(pair)) problems.Add("duplicate lineage pair " + pair);
        if (d.produceTier &lt; 1 || d.produceTier &gt; MergeSim.TopTier)
            problems.Add("dex " + d.id + " produceTier out of 1..TopTier");
        if (d.creatureTier &lt; 1 || d.creatureTier &gt; MergeSim.TopTier)
            problems.Add("dex " + d.id + " creatureTier out of 1..TopTier");
        if (d.chance &lt;= 0f || d.chance &gt; 1f)
            problems.Add("dex " + d.id + " chance out of (0,1]");
        if (!ColorHexParses(d.colorHex))
            problems.Add("dex " + d.id + " colorHex does not parse");
    }
    // U153-D04 W1-b second domain: 12 slots, one per (baseSpecies, baseTier)
    // identity (2 lines x tiers 0..TopTier), unique ids, full coverage.
    var secondIds = new HashSet&lt;string&gt;();
```

```
            var secondPairs = new HashSet&lt;string&gt;();
            if (SecondDex == null || SecondDex.Length == 0)
            {
                problems.Add("secondDex section missing");
            }
            else
            {
                foreach (var d in SecondDex)
                {
                    if (string.IsNullOrEmpty(d.id)) problems.Add("secondDex entry missing id");
                    if (!secondIds.Add(d.id)) problems.Add("duplicate secondDex id " + d.id);
                    if (d.baseSpecies != 0 &amp;&amp; d.baseSpecies != 1)
                        problems.Add("secondDex " + d.id + " baseSpecies must be 0/1, got " + d.baseSpecies);
                    if (d.baseTier &lt; 0 || d.baseTier &gt; MergeSim.TopTier)
                        problems.Add("secondDex " + d.id + " baseTier out of 0..TopTier");
                    string pair = d.baseSpecies + "x" + d.baseTier;
                    if (!secondPairs.Add(pair)) problems.Add("duplicate secondDex base pair " + pair);
                }
                for (int species = 0; species &lt;= 1; species++)
                    for (int tier = 0; tier &lt;= MergeSim.TopTier; tier++)
                        if (!secondPairs.Contains(species + "x" + tier))
                            problems.Add("secondDex missing base pair " + species + "x" + tier);
            }
            // Graybox parity anchor: the sim hatches the TopTier x TopTier pair
            // deterministically, so the data must keep that pair at chance 1.0.
            var top = HybridRule(MergeSim.TopTier, MergeSim.TopTier);
            if (top == null) problems.Add("lineage missing TopTier x TopTier pair");
            else if (top.chance &lt; 1f) problems.Add("TopTier pair chance must stay 1.0 (graybox parity)");
            if (Economy == null)
            {
                problems.Add("economy section missing");
            }
            else
            {
                if (Economy.spawnMinTier &lt; 1 || Economy.spawnMinTier &gt; Economy.spawnMaxTier)
                    problems.Add("spawnMinTier out of window");
                if (Economy.spawnMaxTier &lt; 1 || Economy.spawnMaxTier &gt; MergeSim.TopTier)
                    problems.Add("spawnMaxTier out of 1..TopTier");
            }
            return problems;
        }
        static bool ColorHexParses(string hex)
        {
            if (string.IsNullOrEmpty(hex)) return false;
            Color c;
            return ColorUtility.TryParseHtmlString(hex, out c);
        }
    }
}
// G03MergeHookFlow.cs
```

```
namespace Sy.G03
{
    /// &lt;summary&gt;
    /// Scripted ten-second opening (task S1-T005, red line 11 ten-second hook), the
    /// production port of the E4 graybox MergeHookFlow (attempt2 timeline, evidence
    /// Design/evidence/m6_mergemania_graybox_e4_20260919.md: caption 0.02s, first merge
    /// 0.82s, 3 merges inside 10s, cat-melon 6.33s). Runs on the sim virtual clock at
    /// board open, so the per-second hook assertions stay EditMode-reproducible while
    /// the view renders the same beats the player will keep earning later.
    /// Timeline: T+0.02 caption beat -&gt; T+0.5 guided drop -&gt; ~T+0.8 first merge
    /// (&lt;=1.0s after the drop) -&gt; ~T+2.3 second guided merge -&gt; T+6.3 gifted
    /// cross-family collide -&gt; cat-melon hatch tease (laugh beat included).
    /// &lt;/summary&gt;
    public static class G03MergeHookFlow
    {
        public const string CaptionEventKind = "caption";
        public const string CaptionTextKey = "ui.hook.subtitle";
        public static void Run(MergeSim sim)
        {
            // Pre-seed column 3 bottom-up so the first guided drop completes a merge.
            sim.SeedColumn(3, new[] { 2, 3, 4 }, MergeSpecies.Produce);
            sim.Tick(); // T+0.02: hook caption beat (&lt;=3.0s red-line window)
            sim.EmitExternal(CaptionEventKind, CaptionTextKey);
            sim.AdvanceTo(0.5);
            double firstDropLand = sim.Drop(3, 4, MergeSpecies.Produce); // 4+4 -&gt; 5 watermelon
            sim.EmitExternal("first_drop_landed", "land=" + firstDropLand.ToString("F2"));
            sim.AdvanceTo(2.0);
            sim.SeedColumn(4, new[] { 1, 2, 2 }, MergeSpecies.Produce);
            sim.Drop(4, 2, MergeSpecies.Produce); // second guided merge (2+2 -&gt; 3)
            sim.AdvanceTo(6.0);
            sim.SeedColumn(2, new[] { 5 }, MergeSpecies.Creature); // gifted top-tier cat
            sim.Drop(2, 5, MergeSpecies.Produce);                  // collide -&gt; cat-melon
            sim.AdvanceTo(10.0);
        }
    }
}
// G03Module.cs
using Sy.Core;
namespace Sy.G03
{
    /// &lt;summary&gt;
    /// Production skeleton anchor (S0 slice 1). The Sy.Core touch below is the
    /// consumer canary proving the shared-package wiring resolves: a dangling
    /// asmdef reference never warns on its own - a real using fails loudly.
    /// &lt;/summary&gt;
    public static class G03Module
    {
        public const string ModuleId = "sy.g03.mergemania";
        /// &lt;summary&gt;Resolves a Sy.Core type name; the compile gate fails here if the wiring is broken.&lt;/summar
        public static string CoreHook()
```

```
        {
            return typeof(Logger).FullName;
        }
        /// &lt;summary&gt;S0-T003: touch the i18n loader so the canary covers the text table too.&lt;/summary&gt;
        public static int I18nHook()
        {
            return G03Text.Count();
        }
    }
}
// G03SaveSystem.cs
using System;
using System.Collections.Generic;
using Sy.Core;
namespace Sy.G03
{
    /// &lt;summary&gt;Outcome of a save-slot load (task S2-T001, G02 four-state pattern).&lt;/summary&gt;
    public enum G03LoadStatus
    {
        NewGame,         // slot empty: fresh profile, nothing written yet
        Loaded,          // current-version payload applied as-is
        Migrated,        // older payload upgraded in place (defaults for missing fields)
        CorruptedReset   // unreadable or future-version payload: profile reset to fresh
    }
    /// &lt;summary&gt;
    /// Profile payload for G03 (task S2-T001): discovered dex ids, merge stats, best-chain
    /// record and the opening seed. Pure data holder - the live board snapshot itself is
    /// slice B (board resync), not part of this model.
    /// &lt;/summary&gt;
    public sealed class G03SaveModel
    {
        public List&lt;string&gt; discoveredDex = new List&lt;string&gt;();   // h01..h12 ids the player has hatched
        public List&lt;string&gt; discoveredSecond = new List&lt;string&gt;();   // U153-D04 W1-b2: second-lineage slots
        public int mergeCount;      // lifetime merges (progression stat)
        public int hybridCount;     // lifetime cross-type hybrids (dex fuel)
        public int dropCount;      // lifetime drops (engagement proxy)
        public int bestChain;      // best cascade chain record (bragging stat)
        public int openingSeed;    // seed of the current run's opening (determinism anchor)
        public void Reset()
        {
            discoveredDex = new List&lt;string&gt;();
            discoveredSecond = new List&lt;string&gt;();
            mergeCount = 0;
            hybridCount = 0;
            dropCount = 0;
            bestChain = 0;
            openingSeed = 0;
        }
    }
    /// &lt;summary&gt;
```

```
/// Local-authority persistence for G03 (task S2-T001). Single-key JSON through the
/// Sy.Core save service - red line 12 zero-server doctrine, no cloud functions, no
/// remote hot update. Version skeleton "g03v": a payload without the version key is
/// migrated by filling defaults for missing fields; a payload from a newer build
/// degrades to a fresh profile instead of crashing; corrupted payloads never crash
/// the boot (three-level guard, G02 C052 service-level FormatException lesson).
/// &lt;/summary&gt;
public static class G03SaveSystem
{
    public const string Slot = "g03_main";
    public const string VersionKey = "g03v";
    public const int CurrentVersion = 1;
    /// &lt;summary&gt;Snapshot the live sim counters into the model (dex set stays caller-owned).&lt;/summary&gt;
    public static void Capture(MergeSim sim, G03SaveModel model)
    {
        if (sim == null || model == null) return;
        model.mergeCount = sim.MergeCount;
        model.hybridCount = sim.HybridCount;
        model.dropCount = sim.DropCount;
        model.bestChain = sim.MaxChainInOneCascade;
        model.openingSeed = sim.Seed;
    }
    /// &lt;summary&gt;Serialize the full profile into the single-key JSON payload.&lt;/summary&gt;
    public static string Serialize(G03SaveModel model)
    {
        if (model == null) throw new ArgumentNullException(nameof(model));
        var data = new Dictionary&lt;string, object&gt;
        {
            { VersionKey, (long)CurrentVersion },
            { "merges", (long)model.mergeCount },
            { "hybrids", (long)model.hybridCount },
            { "drops", (long)model.dropCount },
            { "chain", (long)model.bestChain },
            { "seed", (long)model.openingSeed },
        };
        var dex = new List&lt;object&gt;();
        if (model.discoveredDex != null)
        {
            foreach (var id in model.discoveredDex) dex.Add(id);
        }
        data["dex"] = dex;
        // U153-D04 W1-b2: the second-lineage lights ride the same profile payload;
        // older v1 payloads without the key hydrate as an empty list (honest default).
        var dex2 = new List&lt;object&gt;();
        if (model.discoveredSecond != null)
        {
            foreach (var id in model.discoveredSecond) dex2.Add(id);
        }
        data["dex2"] = dex2;
        return MiniJson.Serialize(data);
```

```
}
/// &lt;summary&gt;Write the profile through the save service (local authority).&lt;/summary&gt;
public static void Persist(ISaveService save, G03SaveModel model, string slot = Slot)
{
    if (save == null) return;
    save.Save(slot, Serialize(model));
}
/// &lt;summary&gt;
/// Load the slot into the given model. Never throws on bad data: corrupted or
/// future-version payloads reset the profile to fresh defaults.
/// &lt;/summary&gt;
public static G03LoadStatus LoadInto(ISaveService save, G03SaveModel model, string slot = Slot)
{
    if (save == null) throw new ArgumentNullException(nameof(save));
    if (model == null) throw new ArgumentNullException(nameof(model));
    string raw;
    try
    {
        raw = save.Load(slot);
    }
    catch (Exception e)
    {
        // The service itself chokes on a corrupted payload - degrade, never crash the boot.
        Logger.Warn("G03.Save", "corrupt payload at service level, resetting slot: " + e.Message);
        return Fresh(model);
    }
    if (string.IsNullOrEmpty(raw))
    {
        return G03LoadStatus.NewGame;
    }
    Dictionary&lt;string, object&gt; data;
    try
    {
        data = MiniJson.Deserialize(raw) as Dictionary&lt;string, object&gt;;
    }
    catch (Exception e)
    {
        Logger.Warn("G03.Save", "corrupt payload, resetting slot: " + e.Message);
        return Fresh(model);
    }
    if (data == null)
    {
        Logger.Warn("G03.Save", "non-dict payload, resetting slot");
        return Fresh(model);
    }
    int version = ReadInt(data, VersionKey, 0);
    if (version &gt; CurrentVersion)
    {
        Logger.Warn("G03.Save", "future version " + version + " &gt; " + CurrentVersion + ", resetting slot");
        return Fresh(model);
```

```
            }
            model.mergeCount = ReadInt(data, "merges", 0);
            model.hybridCount = ReadInt(data, "hybrids", 0);
            model.dropCount = ReadInt(data, "drops", 0);
            model.bestChain = ReadInt(data, "chain", 0);
            model.openingSeed = ReadInt(data, "seed", 0);
            model.discoveredDex = ReadStrings(data, "dex");
            model.discoveredSecond = ReadStrings(data, "dex2");   // U153-D04 W1-b2: missing key = empty list
            return version &lt; CurrentVersion ? G03LoadStatus.Migrated : G03LoadStatus.Loaded;
        }
        static G03LoadStatus Fresh(G03SaveModel model)
        {
            model.Reset();
            return G03LoadStatus.CorruptedReset;
        }
        static int ReadInt(Dictionary&lt;string, object&gt; data, string key, int fallback)
        {
            if (!data.TryGetValue(key, out var value) || value == null) return fallback;
            try
            {
                return Convert.ToInt32(Convert.ToDouble(value));
            }
            catch
            {
                return fallback;
            }
        }
        static List&lt;string&gt; ReadStrings(Dictionary&lt;string, object&gt; data, string key)
        {
            var list = new List&lt;string&gt;();
            if (data.TryGetValue(key, out var value) &amp;&amp; value is System.Collections.IEnumerable rows)
            {
                foreach (var row in rows)
                {
                    if (row is string s &amp;&amp; !string.IsNullOrEmpty(s)) list.Add(s);
                }
            }
            return list;
        }
    }
    /// &lt;summary&gt;
    /// Board/session snapshot persistence (task S2-T001 slice B). The live mid-run
    /// situation - columns, waste heat, rotate phase, run counters - lives in its
    /// own slot so the lifetime profile payload (sliceA, g03v v1) stays byte-stable.
    /// Same three-level guard as the profile layer: corrupted or future-version
    /// payloads never crash the boot, they reset the board (the profile keeps the
    /// lifetime stats; the caller just starts a fresh run). Local authority through
    /// the Sy.Core save service - red line 12: zero server, zero network.
    /// &lt;/summary&gt;
    public static class G03BoardSnapshot
```

```
{
    public const string Slot = "g03_board";
    public const string VersionKey = "g03v";
    public const int CurrentVersion = 1;
    public static string Serialize(MergeBoardState state)
    {
        if (state == null) throw new ArgumentNullException(nameof(state));
        var data = new Dictionary&lt;string, object&gt;
        {
            { VersionKey, (long)CurrentVersion },
            { "seed", (long)state.Seed },
            { "time", state.TimeSec },
            { "nextRotate", state.NextRotateAt },
            { "heat", state.Heat },
            { "nextId", (long)state.NextId },
            { "drops", (long)state.DropCount },
            { "merges", (long)state.MergeCount },
            { "chain", (long)state.MaxChainInOneCascade },
            { "hybrids", (long)state.HybridCount },
            { "heatBursts", (long)state.HeatBurstCount },
            { "rotates", (long)state.RotateCount },
            { "rotateChain", (long)state.RotateChainMerges },
            { "overflows", (long)state.OverflowCount },
        };
        var cols = new List&lt;object&gt;();
        if (state.Columns != null)
        {
            foreach (var column in state.Columns)
            {
                var colData = new List&lt;object&gt;();
                if (column != null)
                {
                    foreach (var item in column)
                    {
                        colData.Add(new Dictionary&lt;string, object&gt;
                        {
                            { "t", (long)item.Tier },
                            { "s", (long)(int)item.Species },
                            { "i", (long)item.Id },
                        });
                    }
                }
                cols.Add(colData);
            }
        }
        data["cols"] = cols;
        return MiniJson.Serialize(data);
    }
    public static void Persist(ISaveService save, MergeBoardState state, string slot = Slot)
    {
```

```
        if (save == null) return;
        save.Save(slot, Serialize(state));
    }
    /// &lt;summary&gt;
    /// Load the board slot into the given state bag. Never throws on bad data.
    /// NewGame leaves the caller-supplied state untouched (mirror of the profile
    /// layer); CorruptedReset fills safe fresh-run defaults.
    /// &lt;/summary&gt;
    public static G03LoadStatus LoadInto(ISaveService save, MergeBoardState state, string slot = Slot)
    {
        if (save == null) throw new ArgumentNullException(nameof(save));
        if (state == null) throw new ArgumentNullException(nameof(state));
        string raw;
        try
        {
            raw = save.Load(slot);
        }
        catch (Exception e)
        {
            Logger.Warn("G03.Board", "corrupt payload at service level, resetting board: " + e.Message);
            return Fresh(state);
        }
        if (string.IsNullOrEmpty(raw))
        {
            return G03LoadStatus.NewGame;
        }
        Dictionary&lt;string, object&gt; data;
        try
        {
            data = MiniJson.Deserialize(raw) as Dictionary&lt;string, object&gt;;
        }
        catch (Exception e)
        {
            Logger.Warn("G03.Board", "corrupt payload, resetting board: " + e.Message);
            return Fresh(state);
        }
        if (data == null)
        {
            Logger.Warn("G03.Board", "non-dict payload, resetting board");
            return Fresh(state);
        }
        int version = ReadInt(data, VersionKey, 0);
        if (version &gt; CurrentVersion)
        {
            Logger.Warn("G03.Board", "future version " + version + " &gt; " + CurrentVersion + ", resetting board");
            return Fresh(state);
        }
        state.Seed = ReadInt(data, "seed", 0);
        state.TimeSec = ReadDouble(data, "time", 0.0);
        state.NextRotateAt = ReadDouble(data, "nextRotate", MergeSim.RotatePeriodSec);
```

```
        state.Heat = ReadDouble(data, "heat", 0.0);
        state.NextId = ReadInt(data, "nextId", 1);
        state.DropCount = ReadInt(data, "drops", 0);
        state.MergeCount = ReadInt(data, "merges", 0);
        state.MaxChainInOneCascade = ReadInt(data, "chain", 0);
        state.HybridCount = ReadInt(data, "hybrids", 0);
        state.HeatBurstCount = ReadInt(data, "heatBursts", 0);
        state.RotateCount = ReadInt(data, "rotates", 0);
        state.RotateChainMerges = ReadInt(data, "rotateChain", 0);
        state.OverflowCount = ReadInt(data, "overflows", 0);
        state.Columns = ReadColumns(data);
        // Legacy payloads without a nextId would collide their restored ids with
        // future drops - always continue above the highest restored id.
        int maxId = 0;
        for (int c = 0; c &lt; MergeSim.Cols; c++)
        {
            if (state.Columns[c] == null) continue;
            foreach (var item in state.Columns[c]) if (item.Id &gt; maxId) maxId = item.Id;
        }
        if (maxId + 1 &gt; state.NextId) state.NextId = maxId + 1;
        return version &lt; CurrentVersion ? G03LoadStatus.Migrated : G03LoadStatus.Loaded;
    }
    static G03LoadStatus Fresh(MergeBoardState state)
    {
        state.Seed = 0;
        state.TimeSec = 0.0;
        state.NextRotateAt = MergeSim.RotatePeriodSec;
        state.Heat = 0.0;
        state.NextId = 1;
        state.DropCount = 0;
        state.MergeCount = 0;
        state.MaxChainInOneCascade = 0;
        state.HybridCount = 0;
        state.HeatBurstCount = 0;
        state.RotateCount = 0;
        state.RotateChainMerges = 0;
        state.OverflowCount = 0;
        state.Columns = EmptyColumns();
        return G03LoadStatus.CorruptedReset;
    }
    static List&lt;MergeItem&gt;[] EmptyColumns()
    {
        var cols = new List&lt;MergeItem&gt;[MergeSim.Cols];
        for (int i = 0; i &lt; MergeSim.Cols; i++) cols[i] = new List&lt;MergeItem&gt;();
        return cols;
    }
    static List&lt;MergeItem&gt;[] ReadColumns(Dictionary&lt;string, object&gt; data)
    {
        var cols = EmptyColumns();
        if (!data.TryGetValue("cols", out var value) ||
```

```
                !(value is System.Collections.IEnumerable rows)) return cols;
            int c = 0;
            foreach (var row in rows)
            {
                if (c &gt;= MergeSim.Cols) break;
                if (row is System.Collections.IEnumerable items)
                {
                    foreach (var entry in items)
                    {
                        if (entry is Dictionary&lt;string, object&gt; cell)
                        {
                            cols[c].Add(new MergeItem
                            {
                                Tier = ReadInt(cell, "t", 1),
                                Species = (MergeSpecies)ReadInt(cell, "s", 0),
                                Id = ReadInt(cell, "i", 0),
                            });
                        }
                    }
                }
                c++;
            }
            return cols;
        }
        static int ReadInt(Dictionary&lt;string, object&gt; data, string key, int fallback)
        {
            if (!data.TryGetValue(key, out var value) || value == null) return fallback;
            try
            {
                return Convert.ToInt32(Convert.ToDouble(value));
            }
            catch
            {
                return fallback;
            }
        }
        static double ReadDouble(Dictionary&lt;string, object&gt; data, string key, double fallback)
        {
            if (!data.TryGetValue(key, out var value) || value == null) return fallback;
            try
            {
                return Convert.ToDouble(value);
            }
            catch
            {
                return fallback;
            }
        }
    }
}
```

```
// G03Sfx.cs
using System;
using System.Collections.Generic;
using Sy.Core;
using UnityEngine;
using Logger = Sy.Core.Logger;
namespace Sy.G03
{
    /// &lt;summary&gt;
    /// Procedural sfx + signature leitmotif for G03 (task S1-T005, X028 earworm red line:
    /// hook hummable within 3s, unique across the portfolio, zero shipped audio files -
    /// GDD zero-audio-file doctrine, quota-mode legal). Pure math over float[] buffers:
    /// deterministic, EditMode testable, no Random, no files.
    ///
    /// G03 signature hook "jelly triple descend + hatch pop bell": three soft jelly
    /// wobble notes stepping DOWN a semitone each (A5 880.0 -&gt; G#5 830.6 -&gt; G5 784.0),
    /// a short breath, then the hatch pop + bright bell (the cat-melon being born).
    /// It mirrors the merge fantasy (two things squish into a new one) and inverts the
    /// G02 rising wind-up contour, so the two hooks stay distinct at a glance listen.
    /// The same jelly note feeds the per-merge feedback so the signature is heard
    /// organically inside the opening (three merges = three descending steps).
    /// &lt;/summary&gt;
    public static class G03Sfx
    {
        public const int SampleRate = 22050;
        public const string JellyPlop = "plop";     // jelly wobble note (drop land / merge)
        public const string HatchBell = "hatch";    // hatch pop + bell (hybrid cat-melon)
        public const string Leitmotif = "leitmotif"; // signature hook (opening beat)
        public const string ChainPlopPrefix = "chain_plop_"; // combo ladder (task S2-T003, GAP-B)
        public const int ChainPlopMaxStep = 12;     // one octave up (C5 -&gt; C6), then holds the peak
        public const string DexLight = "dex_light"; // dex light-up sparkle (task S2-T003 collection beat)
        static readonly Dictionary&lt;string, AudioClip&gt; ClipCache = new();
        static AudioSource _host;
        static AudioSource _bgm;
        /// &lt;summary&gt;Deterministic sample synthesis for one clip id (pure, no Unity objects).&lt;/summary&gt;
        public static float[] Synth(string clipId)
        {
            switch (clipId)
            {
                case JellyPlop: return PlopSamples();
                case HatchBell: return HatchSamples();
                case Leitmotif: return LeitmotifSamples();
                case DexLight: return DexLightSamples();
                default:
                    if (clipId != null &amp;&amp; clipId.StartsWith(ChainPlopPrefix, StringComparison.Ordinal))
                        return PlopSamples(523.25 * Math.Pow(2.0, ChainStep(clipId) / 12.0));
                    return new float[0];
            }
        }
        /// &lt;summary&gt;Chain ladder clip id for a merge step (S2-T003 GAP-B); step clamps to 0..max.&lt;/summary&gt;
```

```
public static string ChainPlopId(int step)
{
    if (step &lt; 0) step = 0;
    if (step &gt; ChainPlopMaxStep) step = ChainPlopMaxStep;
    return ChainPlopPrefix + step;
}
static int ChainStep(string clipId)
{
    int v;
    return int.TryParse(clipId.Substring(ChainPlopPrefix.Length), out v)
        ? Math.Min(Math.Max(v, 0), ChainPlopMaxStep)
        : 0;
}
/// &lt;summary&gt;Cached AudioClip wrapper for playback (main thread only).&lt;/summary&gt;
public static AudioClip Clip(string clipId)
{
    if (ClipCache.TryGetValue(clipId, out var cached) &amp;&amp; cached != null) return cached;
    var data = Synth(clipId);
    if (data.Length == 0) return null;
    var clip = AudioClip.Create("g03_sfx_" + clipId, data.Length, 1, SampleRate, false);
    clip.SetData(data, 0);
    ClipCache[clipId] = clip;
    return clip;
}
/// &lt;summary&gt;Fire-and-forget playback on a hidden host; never throws in headless runs.&lt;/summary&gt;
public static void Play(string clipId)
{
    try
    {
        if (_host == null)
        {
            var go = new GameObject("G03SfxHost");
            UnityEngine.Object.DontDestroyOnLoad(go);
            _host = go.AddComponent&lt;AudioSource&gt;();
        }
        var clip = Clip(clipId);
        if (clip != null) _host.PlayOneShot(clip);
    }
    catch (Exception e)
    {
        Logger.Warn("G03.Sfx", "play skipped (" + clipId + "): " + e.Message);
    }
}
/// &lt;summary&gt;BGM channel, reserved for the manifest bgm row
/// (g03_bgm_main, AI-audio transition of the zero-file doctrine):
/// looping, null-safe, honest false while the asset is not in -
/// wiring once, zero rework on arrival.&lt;/summary&gt;
public static bool AttachBgm(AudioClip clip)
{
    try
```

```
        {
            if (clip == null) return false;
            if (_bgm == null)
            {
                if (_host == null)
                {
                    var go = new GameObject("G03SfxHost");
                    UnityEngine.Object.DontDestroyOnLoad(go);
                    _host = go.AddComponent&lt;AudioSource&gt;();
                }
                _bgm = _host.gameObject.AddComponent&lt;AudioSource&gt;();
                _bgm.loop = true;
                _bgm.playOnAwake = false;
            }
            _bgm.clip = clip;
            _bgm.Play();
            return true;
        }
        catch (Exception e)
        {
            Logger.Warn("G03.Sfx", "bgm attach skipped: " + e.Message);
            return false;
        }
    }
    /// &lt;summary&gt;Boot-time BGM loader (X027): resolves the manifest bgm
    /// row through the audio catalog and attaches it. Honest no-op while
    /// the imported clip or the catalog asset is not in yet, so headless
    /// checks stay green on any project state.&lt;/summary&gt;
    public static bool BootBgm()
    {
        return AttachBgm(AudioRuntime.Get("g03_bgm_main"));
    }
    // ---- synthesis recipes ---------------------------------------------------
    /// &lt;summary&gt;Jelly wobble note: 523.3Hz (C5) sine with a slow 6Hz wobble, plucked decay.&lt;/summary&gt;
    static float[] PlopSamples()
    {
        return PlopSamples(523.25);
    }
    /// &lt;summary&gt;
    /// Transposed jelly wobble note (S2-T003 combo ladder): same recipe at a
    /// multiplied base frequency, so every ladder step keeps the exact plop
    /// timbre - the ascending run stays inside the signature family while its
    /// contour (up) stays mutually exclusive with the leitmotif's descent.
    /// &lt;/summary&gt;
    static float[] PlopSamples(double baseFreq)
    {
        int n = (int)(SampleRate * 0.16);
        var buf = new float[n];
        for (int i = 0; i &lt; n; i++)
        {
```

```
            double t = (double)i / SampleRate;
            double attack = t &lt; 0.008 ? t / 0.008 : 1.0;              // tiny soft attack, no click
            double env = attack * Math.Exp(-t / 0.055);
            double wobble = 1.0 + 0.03 * Math.Sin(2.0 * Math.PI * 6.0 * t); // jelly quiver
            buf[i] = (float)(Math.Sin(2.0 * Math.PI * baseFreq * wobble * t) * env);
        }
        return Normalize(buf, 0.70);
    }
    /// &lt;summary&gt;Hatch pop + bell: 40ms pitch-up chirp (the pop), then a bright E6 bell decay.&lt;/summary&gt;
    static float[] HatchSamples()
    {
        const double popLen = 0.06;
        const double bellLen = 0.55;
        int total = (int)(SampleRate * (popLen + bellLen));
        var buf = new float[total];
        int popN = (int)(SampleRate * popLen);
        for (int i = 0; i &lt; popN; i++)
        {
            double t = (double)i / SampleRate;
            double freq = 300.0 + 1400.0 * (t / popLen);               // rising chirp = the "bo"
            double env = Math.Sin(Math.PI * t / popLen);               // smooth pop envelope
            buf[i] = (float)(Math.Sin(2.0 * Math.PI * freq * t) * env * 0.8);
        }
        for (int i = 0; popN + i &lt; total; i++)
        {
            double t = (double)i / SampleRate;
            double env = Math.Exp(-t / 0.22);
            double e6 = Math.Sin(2.0 * Math.PI * 1318.5 * t);
            double sparkle = Math.Sin(2.0 * Math.PI * 2637.0 * t) * Math.Exp(-t / 0.08) * 0.35;
            buf[popN + i] = (float)((e6 * 0.85 + sparkle) * env);
        }
        return Normalize(buf, 0.78);
    }
    /// &lt;summary&gt;
    /// Signature leitmotif "jelly triple descend + hatch pop bell": A5 -&gt; G#5 -&gt; G5
    /// jelly notes (semitone steps down), breath, then the full hatch pop bell.
    /// ~1.26s total, inside the 3s hummable red line.
    /// &lt;/summary&gt;
    static float[] LeitmotifSamples()
    {
        const double noteLen = 0.15;
        const double breath = 0.09;
        const double tailLen = 0.61; // hatch pop (0.06) + bell decay (0.55)
        int total = (int)(SampleRate * (noteLen * 3.0 + breath + tailLen));
        var buf = new float[total];
        double[] steps = { 880.0, 830.61, 784.0 };                     // semitone descend
        for (int s = 0; s &lt; 3; s++)
        {
            int start = (int)(SampleRate * (noteLen * s));
            int len = (int)(SampleRate * noteLen);
```

```
            for (int i = 0; i &lt; len &amp;&amp; start + i &lt; total; i++)
            {
                double t = (double)i / SampleRate;
                double attack = t &lt; 0.008 ? t / 0.008 : 1.0;
                double env = attack * Math.Exp(-t / 0.075);
                double wobble = 1.0 + 0.03 * Math.Sin(2.0 * Math.PI * 6.0 * t);
                buf[start + i] = (float)(Math.Sin(2.0 * Math.PI * steps[s] * wobble * t) * env);
            }
        }
        int tailStart = (int)(SampleRate * (noteLen * 3.0 + breath));
        int popN = (int)(SampleRate * 0.06);
        for (int i = 0; i &lt; popN &amp;&amp; tailStart + i &lt; total; i++)
        {
            double t = (double)i / SampleRate;
            double freq = 300.0 + 1400.0 * (t / 0.06);
            double env = Math.Sin(Math.PI * t / 0.06);
            buf[tailStart + i] = (float)(Math.Sin(2.0 * Math.PI * freq * t) * env * 0.8);
        }
        for (int i = 0; tailStart + popN + i &lt; total; i++)
        {
            double t = (double)i / SampleRate;
            double env = Math.Exp(-t / 0.22);
            double e6 = Math.Sin(2.0 * Math.PI * 1318.5 * t);
            double sparkle = Math.Sin(2.0 * Math.PI * 2637.0 * t) * Math.Exp(-t / 0.08) * 0.35;
            buf[tailStart + popN + i] = (float)((e6 * 0.85 + sparkle) * env);
        }
        return Normalize(buf, 0.80);
    }
    /// &lt;summary&gt;
    /// Dex light-up sparkle (task S2-T003): two bright bell notes stepping UP a
    /// major third (E6 -&gt; G#6) - the collection moment brightens. Same bell family
    /// as the hatch so the discovery reads as the hybrid's reward echo, not a new
    /// instrument; the ascending contour stays mutually exclusive with the
    /// descending signature leitmotif (X028 hook discipline).
    /// &lt;/summary&gt;
    static float[] DexLightSamples()
    {
        double[] steps = { 1318.51, 1661.22 };                        // E6 -&gt; G#6 = light goes ON
        const double noteLen = 0.18;
        const double gapLen = 0.045;
        const double ringLen = 0.32;
        int total = (int)(SampleRate * (noteLen * 2.0 + gapLen + ringLen));
        var buf = new float[total];
        for (int s = 0; s &lt; 2; s++)
        {
            int start = (int)(SampleRate * (noteLen + gapLen) * s);
            int len = (int)(SampleRate * noteLen);
            for (int i = 0; i &lt; len &amp;&amp; start + i &lt; total; i++)
            {
                double t = (double)i / SampleRate;
```

```
                    double attack = t &lt; 0.006 ? t / 0.006 : 1.0;          // soft attack, no click
                    double env = attack * Math.Exp(-t / 0.09);
                    double tone = Math.Sin(2.0 * Math.PI * steps[s] * t);
                    double sparkle = Math.Sin(2.0 * Math.PI * steps[s] * 2.0 * t) * Math.Exp(-t / 0.05) * 0.3;
                    buf[start + i] = (float)((tone * 0.85 + sparkle) * env);
                }
            }
            int tailStart = (int)(SampleRate * (noteLen * 2.0 + gapLen));
            for (int i = 0; tailStart + i &lt; total; i++)
            {
                double t = (double)i / SampleRate;
                double env = Math.Exp(-t / 0.12);
                double g6 = Math.Sin(2.0 * Math.PI * 1661.22 * t);
                double sparkle = Math.Sin(2.0 * Math.PI * 3322.44 * t) * Math.Exp(-t / 0.06) * 0.35;
                buf[tailStart + i] = (float)((g6 * 0.8 + sparkle) * env);
            }
            return Normalize(buf, 0.74);
        }
        /// &lt;summary&gt;Peak normalization to a fixed target - keeps the loudness family tight.&lt;/summary&gt;
        static float[] Normalize(float[] buf, double targetPeak)
        {
            double peak = 0.0;
            for (int i = 0; i &lt; buf.Length; i++)
            {
                double a = Math.Abs(buf[i]);
                if (a &gt; peak) peak = a;
            }
            if (peak &lt; 1e-6) return buf;
            double scale = targetPeak / peak;
            for (int i = 0; i &lt; buf.Length; i++) buf[i] = (float)(buf[i] * scale);
            return buf;
        }
    }
}
// G03Text.cs
using System;
using System.Collections.Generic;
using UnityEngine;
using Logger = Sy.Core.Logger;
namespace Sy.G03
{
    [Serializable]
    public sealed class G03I18nEntry
    {
        public string key;
        public string text;
    }
    [Serializable]
    public sealed class G03I18nFile
    {
```

```
    public G03I18nEntry[] entries;
}
/// &lt;summary&gt;
/// Project i18n table (task S0-T003, red line 4): all player-facing Chinese text is
/// externalized into Resources/cfg_g03_i18n.json. Missing key falls back to the key
/// itself so the game never crashes on text lookup. Mirrors the G02Text paradigm;
/// the coverage assert lands with the S0-T004 gate upgrade (BootstrapG03).
/// &lt;/summary&gt;
public static class G03Text
{
    static Dictionary&lt;string, string&gt; _map;
    public static void Load()
    {
        _map = new Dictionary&lt;string, string&gt;();
        var asset = Resources.Load&lt;TextAsset&gt;("cfg_g03_i18n");
        if (asset == null)
        {
            Logger.Warn("G03.I18n", "cfg_g03_i18n.json not found under Resources");
            return;
        }
        var file = JsonUtility.FromJson&lt;G03I18nFile&gt;(asset.text);
        if (file == null || file.entries == null)
        {
            Logger.Warn("G03.I18n", "cfg_g03_i18n.json failed to parse");
            return;
        }
        foreach (var e in file.entries)
        {
            if (!string.IsNullOrEmpty(e.key)) _map[e.key] = e.text;
        }
    }
    static void EnsureLoaded()
    {
        if (_map == null) Load();
    }
    public static string Get(string key)
    {
        EnsureLoaded();
        if (_map.TryGetValue(key, out var text) &amp;&amp; !string.IsNullOrEmpty(text)) return text;
        Logger.Warn("G03.I18n", "missing i18n key (fallback to key): " + key);
        return key;
    }
    public static bool Has(string key)
    {
        EnsureLoaded();
        return _map.TryGetValue(key, out var text) &amp;&amp; !string.IsNullOrEmpty(text);
    }
    /// &lt;summary&gt;Loaded entry count; the T004 coverage gate asserts against this.&lt;/summary&gt;
    public static int Count()
    {
```

```
            EnsureLoaded();
            return _map != null ? _map.Count : 0;
        }
    }
}
// MergeSim.cs
using System;
using System.Collections.Generic;
namespace Sy.G03
{
    // S1-T001: production core, ported verbatim from the E4 graybox
    // (_Graybox/Scripts/MergeSim.cs, namespace Sy.MergeMania, attempt2 semantics
    // 2026-09-19 - evidence Design/evidence/m6_mergemania_graybox_e4_20260919.md,
    // 13/13 green). The graybox copy stays untouched as the E4 evidence anchor;
    // MergeSimParityTests pin this port to the attempt2 tuning and event semantics.
    // Pure C# by design: physics-free grid cascade model with a fixed-tick virtual
    // clock, fully deterministic under a fixed seed so headless bot-play and
    // red-line-11 per-second hook assertions are reproducible.
    // Model fidelity note: Suika-style physics is intentionally out of scope here;
    // the S1 view layer will add perception beats on top of this deterministic core.
    public enum MergeSpecies { Produce = 0, Creature = 1, Hybrid = 2 }
    public sealed class MergeItem
    {
        public int Tier;
        public MergeSpecies Species;
        public int Id;
    }
    public sealed class SimEvent
    {
        public double TimeSec;   // virtual seconds when the event fired
        public string Kind;      // stable event key: caption/drop/merge/chain3/hybrid/heat_burst/rotate/rotate_chain/la
        public string Detail;   // payload text (English only, source rule)
    }
    /// &lt;summary&gt;
    /// Exact mid-run board state for save/resync (task S2-T001 slice B): all columns,
    /// waste heat, rotate phase and the run counters. Pure data bag - the save layer
    /// (G03BoardSnapshot) serializes it and MergeSim.FromBoardState restores from it.
    /// The rng stream is intentionally NOT part of the state: System.Random internals
    /// are not portable, so a restored sim restarts the spawn stream from the opening
    /// seed. Documented resync semantics: board, heat, rotate phase and counters are
    /// exact; future spawn rolls continue from the seed anchor, which keeps every
    /// run seed-deterministic for red-line-11 replay and headless bots.
    /// &lt;/summary&gt;
    public sealed class MergeBoardState
    {
        public int Seed;
        public double TimeSec;
        public double NextRotateAt;
        public double Heat;
        public int NextId;
```

```
    public int DropCount;
    public int MergeCount;
    public int MaxChainInOneCascade;
    public int HybridCount;
    public int HeatBurstCount;
    public int RotateCount;
    public int RotateChainMerges;
    public int OverflowCount;
    public List&lt;MergeItem&gt;[] Columns;   // per column, index 0 = floor
    public int BurstStrikes;            // U153-D01 slice b: strike count at capture time
    public bool RunOver;                // U153-D01 slice b: terminal judgment latch at capture time
    public MergeRunRecords Records;     // U153-D01 slice b: lifetime records (null = legacy snapshot, records start
    public int RotateManualCount;       // U153-D02 slice b: manual decision-seat counter at capture time
    public double NextManualRotateAt;   // U153-D02 slice b: cooldown phase at capture time (0 = legacy snapshot, se
    public int VentCount;              // U153-D03 slice b: active vent decision-seat counter at capture time (0 = l
}
/// &lt;summary&gt;
/// U153-D01 slice b: lifetime records that survive the soft restart (combo record +
/// dex progress face). Soft restart clears the board and the per-run counters but
/// keeps these - the self-transcendence loop (beat your own best) is the whole
/// point of the soft, non-wipe terminal settlement.
/// &lt;/summary&gt;
public sealed class MergeRunRecords
{
    public int BestMaxChainInOneCascade;   // historical combo record (taskbook: combo record preserved)
    public int BestTierProduce;            // dex progress: species-line top tiers
    public int BestTierCreature;
    public int BestTierHybrid;             // hybrid line top (0 = undiscovered)
    public int LifetimeHybridCount;        // hybrid discoveries (dex milestone face)
    public int LifetimeMergeCount;         // total merges across runs
    public int RunsPlayed;                 // soft restarts consumed (run count)
    public int SeasonIndex;                // U153-D04 W2: device-clock ISO week the season record last folded (0 =
    public int SeasonBestChain;            // U153-D04 W2: best chain inside the current season (week rollover resta
    public int SecondLineageUnlocked;      // U153-D04 W1-a: main dex 12/12 unlock latch (pure derivation, one-way)
    public int SecondLineageAlbumCount;    // U153-D04 W1-a: second-lineage lit slots (W1-b lights them; 0 = honest
}
public sealed class MergeSim
{
    public const int Cols = 7;
    public const int Rows = 10;
    public const int TopTier = 5;                // watermelon (produce) / cat (creature) line top
    public const double TickSec = 1.0 / 60.0;
    public const double FallSec = 0.3;           // perceptible fall beat before settle feedback
    public const double RotatePeriodSec = 30.0;   // board rotation cadence (pitch mechanic #2)
    // U153-D02 slice b (C992 spec): manual early-rotate decision seat - the cooldown
    // window between player-requested turns. C1000 slice c lock: TuneSweepRotateCalib
    // band-median pick 8.0 (manual pooled 0.667 in-band, mph 0.769 &gt; passive 0.758,
    // fewest manual overflows; the 10.0 pre-calib default measured survival-negative
    // vs passive). Strike ladder + heat cap stay production identity.
    public const double RotateCooldownSec = 8.0;
```

```
// U153-D01 slice d-p4 value lock (C990): calibrated 6.0 -&gt; 1.40 - the
// qualifying cell of the d-p3c heatPerMerge grid (pooled 0.667 in
// [0.50,0.95], F1-verbatim sessions; the 6.0 E4-era estimate kept the
// valve punishing engagement at pooled 0.333). Strike ladder + heat
// cap stay production identity; the graybox E4 anchors are untouched.
public const double HeatPerMerge = 1.40;      // waste-heat gauge push per merge unit (pitch mechanic #3)
public const double HeatCap = 100.0;
// U153-D03 slice c value lock (C1026): calibrated 12.0 -&gt; 6.0 - the only
// qualifying cell of the vent refund grid (vent pooled 0.889 in [0.50,0.95]
// with the mph cost face visible, F1-verbatim sessions; the 12.0 pre-calib
// default read 1.000 all-win = out of band - at 12.0 the seat fully defused
// the valve). Passive pooled 0.667 matches the C990 F1 anchor. Strike
// ladder + heat cap stay production identity; E4 anchors untouched.
public const double VentHeatRefund = 6.0;
public const int BurstStrikeCap = 3;         // U153-D01 slice b: third heat burst latches the terminal run-over
// U153-D01 slice c: difficulty potential (F5 face) - the terminal thresholds
// escalate with the player's HISTORICAL best chain on an E1..E4 ladder.
// Run-start tier lock: the tier derives from the records ENTERING the run
// (ctor / SoftRestart), so a fresh sim stays E1 identity for its whole first
// run and the E4 byte-parity anchors (TuneSweep corner: 200 hands, seed 4242,
// merges re-anchored 133 -&gt; 138 by the C1024 OOM cascade fix) keep playing
// byte-identical (judgment-latch law). Values below are
// pre-calibration defaults; slice d (bot TuneSweep extended axes) locks them
// into cfg economy - mechanism first, numbers later (G02 D01 tuning-window
// paradigm, taskbook: no invented thresholds).
public const int DifficultyTierCount = 4;
static readonly int[] BestChainTierLadder = { 4, 6, 8 };                    // E2/E3/E4 entry bounds on the hist
static readonly int[] StrikeCapLadder = { BurstStrikeCap, 4, 5, 6 };        // E1..E4 strictly increasing (E1 =
static readonly double[] HeatCapScaleLadder = { 1.0, 1.10, 1.20, 1.30 };    // E1..E4 strictly increasing (E1 =
// U153-D01 slice d-p3b: runtime tuning hook for the strike-cap ladder.
// The d-p3a grid swapped this ladder via reflection (FieldInfo.SetValue
// on a static readonly array) and the override silently no-oped in this
// Mono runtime - strikes stayed 18 across the whole axis. The calibration
// harness now sets this hook instead. Default null = production ladder,
// byte-identical; only Editor calibration/tests ever set it, and the
// harness self-verifies the hook took effect before firing.
public static int[] StrikeCapLadderOverride;
private readonly List&lt;MergeItem&gt;[] _grid;    // per column, index 0 = floor
private readonly List&lt;SimEvent&gt; _events = new List&lt;SimEvent&gt;();
private readonly Random _rng;
private readonly double _rotatePeriodSec;   // S3-T002 calibration override (defaults = E4 anchors)
private readonly double _heatCap;
private readonly double _heatPerMerge;
private double _timeSec;
private double _nextRotateAt;
private double _nextManualRotateAt;   // U153-D02 slice b: earliest next manual rotate (cooldown phase)
private readonly double _rotateCooldownSec;   // U153-D02 slice c: manual-rotate cooldown override for the bot A
private readonly double _ventHeatRefund;      // U153-D03 slice c: vent refund override for the bot A/B sweep (d
private double _heat;
private int _nextId = 1;
```

```
// U153-D01 slice b: three-strike terminal state + revive seat.
private int _burstStrikes;
private bool _runOver;
private MergeBoardState _reviveSnapshot;   // exact pre-fatal-burst capture (U149 revive: precise recovery)
private int _difficultyTier;               // U153-D01 slice c: run-start tier lock (0 = E1 .. 3 = E4)
// U153-D03 slice b fix (C1024): resolve re-entrancy latch + deferred burst
// queue. AddHeat during ResolveColumn queues the burst instead of firing it
// mid-chain; the resolve exit drains the queue through the identical burst
// body. Decompose items can no longer feed the pass that spawned them, which
// cuts the self-sustaining merge -&gt; burst -&gt; decompose -&gt; merge cascade
// (C1023 OOM diagnosis, Design/evidence/g03_d03b_gate_c1023_20260924).
private bool _inResolveColumn;
private int _pendingBursts;
public int DropCount { get; private set; }
public int MergeCount { get; private set; }
public int MaxChainInOneCascade { get; private set; }
public int HybridCount { get; private set; }
// U153-D04 W1-b: run-scoped hybrid x base fusion counter (observability for
// the second-lineage lighting engine; the persistent face is the presenter
// fold into Records.SecondLineageAlbumCount, not this counter).
public int SecondFusionCount { get; private set; }
public int HeatBurstCount { get; private set; }
public int RotateCount { get; private set; }
public int RotateChainMerges { get; private set; }
public int RotateManualCount { get; private set; }   // U153-D02 slice b: manual decision-seat counter (RotateCo
public int VentCount { get; private set; }           // U153-D03 slice b: active vent decision-seat counter (Hea
public int OverflowCount { get; private set; }
public int Seed { get; private set; }   // opening seed (save anchor, task S2-T001)
// U153-D01 slice b: terminal run-over face (F1 probe family). Judgment latch,
// NOT a mechanical lock: the sim keeps simulating if pushed so the E4
// byte-parity bot anchors stay identical; the view owns stopping the run.
public int BurstStrikes { get { return _burstStrikes; } }
public bool IsRunOver { get { return _runOver; } }
public bool HasReviveOffer { get { return _runOver &amp;&amp; _reviveSnapshot != null; } }
public MergeRunRecords Records { get; private set; }
// ---- U153-D01 slice c: difficulty potential (F5 probe faces) ----
/// &lt;summary&gt;F5 face: historical-best-chain tier ladder, E1..E4 as 0..3
/// (monotone step over BestChainTierCurveOf bounds).&lt;/summary&gt;
public static int BestChainTierOf(int bestChain)
{
    int tier = 0;
    for (int i = 0; i &lt; BestChainTierLadder.Length; i++)
        if (bestChain &gt;= BestChainTierLadder[i]) tier = i + 1;
    return tier;
}
/// &lt;summary&gt;F5 face: the tier boundary ladder (entry bounds on the
/// historical best chain, length = DifficultyTierCount - 1).&lt;/summary&gt;
public static int[] BestChainTierCurveOf()
{
    return (int[])BestChainTierLadder.Clone();
```

```
}
/// &lt;summary&gt;F5 face: this run's difficulty tier (run-start lock, 0 = E1..3 = E4).&lt;/summary&gt;
public int DifficultyScaling()
{
    return _difficultyTier;
}
/// &lt;summary&gt;Strike capacity at a tier (E1..E4 strictly increasing).
/// Reads the d-p3b tuning hook when set; null = production ladder.&lt;/summary&gt;
public static int BurstStrikeCapAtTier(int tier)
{
    int[] ladder = StrikeCapLadderOverride ?? StrikeCapLadder;
    if (tier &lt; 0) tier = 0;
    if (tier &gt;= ladder.Length) tier = ladder.Length - 1;
    return ladder[tier];
}
/// &lt;summary&gt;Heat cap at a tier for this sim's base cap (E1..E4 strictly
/// increasing; E1 = base identity, so calibrated ctors keep their anchors).&lt;/summary&gt;
public double HeatCapAtTier(int tier)
{
    if (tier &lt; 0) tier = 0;
    if (tier &gt;= HeatCapScaleLadder.Length) tier = HeatCapScaleLadder.Length - 1;
    return _heatCap * HeatCapScaleLadder[tier];
}
/// &lt;summary&gt;This run's live strike cap (run-start tier lock).&lt;/summary&gt;
public int EffectiveBurstStrikeCap { get { return BurstStrikeCapAtTier(_difficultyTier); } }
/// &lt;summary&gt;This run's live heat cap (run-start tier lock).&lt;/summary&gt;
public double EffectiveHeatCap { get { return HeatCapAtTier(_difficultyTier); } }
// S2-T001 sliceB read access: rotate phase (next 90-degree turn boundary).
public double NextRotateAtSec { get { return _nextRotateAt; } }
// S2-T001 sliceB read access: next item id (id continuity across resync).
public int NextId { get { return _nextId; } }
public double TimeSec { get { return _timeSec; } }
public double Heat { get { return _heat; } }
public int ItemCount { get { int n = 0; for (int i = 0; i &lt; Cols; i++) n += _grid[i].Count; return n; } }
public IReadOnlyList&lt;SimEvent&gt; Events { get { return _events; } }
public MergeSim(int seed) : this(seed, RotatePeriodSec, HeatCap, HeatPerMerge) { }
public MergeSim(int seed, double rotatePeriodSec, double heatCap, double heatPerMerge)
    : this(seed, rotatePeriodSec, heatCap, heatPerMerge, 0, 0) { }
/// &lt;summary&gt;
/// S3-T002 calibration ctor: per-run economy overrides for the bot tune-sweep.
/// Non-positive values fall back to the E4-anchored constants. The parameterless
/// defaults stay the E4 parity anchors (MergeSimParityTests pin them), and
/// FromBoardState resync keeps the default cadence on purpose: calibrated values
/// are persisted in cfg_g03_catalog economy (slice B), never in save blobs.
/// U153-D02 slice c: the fifth parameter overrides the manual-rotate cooldown
/// window for the D02 A/B sweep (non-positive = the RotateCooldownSec const;
/// the RequestRotate gate reads the field, so every default path stays
/// byte-identical).
/// U153-D03 slice c: the sixth parameter overrides the vent heat refund
/// for the D03 A/B sweep (non-positive = the VentHeatRefund const; the
```

```
            PlayerPrefs.SetString(SaveService.Prefix + BoardSlot, legacy);
            var save = new SaveService();
            var state = new MergeBoardState();
            var status = G03BoardSnapshot.LoadInto(save, state, BoardSlot);
            Assert.That(status, Is.EqualTo(G03LoadStatus.Migrated));
            Assert.That(state.Columns[0].Count, Is.EqualTo(1));
            Assert.That(state.Columns[0][0].Tier, Is.EqualTo(2));
            Assert.That(state.Columns[0][0].Id, Is.EqualTo(7));
            Assert.That(state.MergeCount, Is.EqualTo(0), "missing counters default to zero");
            Assert.That(state.NextId, Is.EqualTo(8), "next id continues above restored ids");
            Assert.That(state.TimeSec, Is.EqualTo(0.0));
        }
        [Test]
        public void BoardFutureVersionPayload_DegradesToFresh()
        {
            string future = MiniJson.Serialize(new Dictionary&lt;string, object&gt;
            {
                { G03BoardSnapshot.VersionKey, (long)99 },
                { "merges", (long)9 },
                { "cols", new List&lt;object&gt;() },
            });
            PlayerPrefs.SetString(SaveService.Prefix + BoardSlot, future);
            var save = new SaveService();
            var state = new MergeBoardState();
            var status = G03BoardSnapshot.LoadInto(save, state, BoardSlot);
            Assert.That(status, Is.EqualTo(G03LoadStatus.CorruptedReset));
            Assert.That(state.MergeCount, Is.EqualTo(0));
            Assert.That(state.NextId, Is.EqualTo(1));
        }
        [Test]
        public void BoardNewGame_EmptySlot_LeavesCallerStateUntouched()
        {
            var save = new SaveService();
            var state = new MergeBoardState { Seed = 4242, TimeSec = 3.5, NextId = 12 };
            var status = G03BoardSnapshot.LoadInto(save, state, BoardSlot);
            Assert.That(status, Is.EqualTo(G03LoadStatus.NewGame));
            Assert.That(state.Seed, Is.EqualTo(4242), "NewGame mirrors the profile layer: caller state untouched");
            Assert.That(state.TimeSec, Is.EqualTo(3.5));
            Assert.That(state.NextId, Is.EqualTo(12));
        }
    }
}
// G03SeasonPercentileTests.cs
using System;
using NUnit.Framework;
using Sy.G03;
namespace Sy.G03.Tests
{
    // U153-D04 polish batch thin seats (W2 + W3, taskbook 03 D04 evaluation
    // archive): the local season seat (device-clock ISO week index + season
```

```
// best chain record surviving the soft restart and the board round-trip)
// and the A3 percentile face (C295 item-2, local bot calibration table -
// AntiDumbGate F1 census, C1028 refire). Wording stays a segmented
// local-rank readout, never a live global population claim (red line 12).
public class G03SeasonPercentileTests
{
    [Test]
    public void SeasonIndexOf_IsoWeekAnchors()
    {
        Assert.AreEqual(1, MergeSim.SeasonIndexOf(new DateTime(2026, 1, 1)), "2026-01-01 Thursday = ISO week 1");
        Assert.AreEqual(1, MergeSim.SeasonIndexOf(new DateTime(2026, 1, 4)), "2026-01-04 Sunday folds onto its week'
        Assert.AreEqual(2, MergeSim.SeasonIndexOf(new DateTime(2026, 1, 5)), "2026-01-05 Monday opens week 2");
        Assert.AreEqual(39, MergeSim.SeasonIndexOf(new DateTime(2026, 9, 24)), "2026-09-24 Thursday = ISO week 39");
        Assert.AreEqual(53, MergeSim.SeasonIndexOf(new DateTime(2026, 12, 31)), "2026 is a 53-week ISO year (Jan 1 T
        Assert.AreEqual(53, MergeSim.SeasonIndexOf(new DateTime(2027, 1, 1)), "2027-01-01 Friday belongs to 2026's w
    }
    [Test]
    public void SeasonFold_TracksBest_And_WeekRolloverRestarts()
    {
        var sim = new MergeSim(42);
        Assert.AreEqual(0, sim.SeasonBestChain, "fresh season record reads zero");
        sim.FoldSeasonChain(39, 4);
        Assert.AreEqual(4, sim.SeasonBestChain, "first fold lands the season best");
        sim.FoldSeasonChain(39, 2);
        Assert.AreEqual(4, sim.SeasonBestChain, "a weaker chain inside the season never lowers the record");
        sim.FoldSeasonChain(39, 6);
        Assert.AreEqual(6, sim.SeasonBestChain, "a stronger chain raises the season record");
        sim.FoldSeasonChain(40, 3);
        Assert.AreEqual(3, sim.SeasonBestChain, "a new ISO week starts a fresh season record");
        // D01 curve reuse: the season best chains into the same tier ladder
        Assert.AreEqual(MergeSim.BestChainTierOf(3), MergeSim.BestChainTierOf(sim.SeasonBestChain),
            "settlement reuses the D01 BestChainTierOf ladder on the season best");
    }
    [Test]
    public void SeasonRecord_Survives_SoftRestart_And_RoundTrip()
    {
        var sim = new MergeSim(42);
        sim.FoldSeasonChain(39, 5);
        sim.SoftRestart();
        Assert.AreEqual(5, sim.SeasonBestChain, "soft restart keeps the season record (records family survives)");
        Assert.AreEqual(39, sim.Records.SeasonIndex, "season index survives the soft restart");
        var state = sim.CaptureBoard();
        var twin = MergeSim.FromBoardState(state);
        Assert.AreEqual(39, twin.Records.SeasonIndex, "season index rides the board round-trip");
        Assert.AreEqual(5, twin.SeasonBestChain, "season best chain rides the board round-trip");
    }
    [Test]
    public void PercentileOfChain_Matches_Locked_Census_Table()
    {
        // C1028 F1 census: per-session maxChain 2,2,2,2,2,3,3,4,5 (9 sessions)
```

```
            Assert.AreEqual(0, MergeSim.PercentileOfChain(0), "below the census floor beats nobody");
            Assert.AreEqual(0, MergeSim.PercentileOfChain(2), "the census floor was reached by every session");
            Assert.AreEqual(56, MergeSim.PercentileOfChain(3), "5/9 census sessions below a 3-chain");
            Assert.AreEqual(78, MergeSim.PercentileOfChain(4), "7/9 census sessions below a 4-chain");
            Assert.AreEqual(89, MergeSim.PercentileOfChain(5), "8/9 census sessions below a 5-chain");
            Assert.AreEqual(100, MergeSim.PercentileOfChain(6), "over the census ceiling");
            Assert.AreEqual(100, MergeSim.PercentileOfChain(99), "ceiling clamps at 100");
            int prev = -1;
            for (int chain = 0; chain &lt;= 12; chain++)
            {
                int p = MergeSim.PercentileOfChain(chain);
                Assert.GreaterOrEqual(p, prev, "percentile is non-decreasing at chain " + chain);
                prev = p;
            }
        }
    }
}
// G03SecondLineageTests.cs
using System;
using System.Collections.Generic;
using NUnit.Framework;
using Sy.Core;
using Sy.G03;
using UnityEngine;
namespace Sy.G03.Tests
{
    // U153-D04 W1-a second-lineage data seat (taskbook 03 section-5 spec anchor):
    // the unlock judgment is a pure derivation off the main dex face (unlock =
    // every cfg dex id discovered), the fold latches the unlock one-way and raises
    // the album count only, and both records survive the soft restart and the
    // board round-trip. The album count stays 0 until the W1-b lighting engine
    // lands - the honest idle state the F3 gate reads as RED (red line 12: zero
    // server, zero wall-clock, zero RNG inside the judgment).
    public class G03SecondLineageTests
    {
        const string SecondSaveSlot = "g03_test_second";
        [TearDown]
        public void TearDown()
        {
            PlayerPrefs.DeleteKey(SaveService.Prefix + SecondSaveSlot);
        }
        static List&lt;string&gt; CatalogDexFace()
        {
            var face = new List&lt;string&gt;();
            var catalog = G03GameCatalog.Load();
            Assert.NotNull(catalog, "cfg_g03_catalog.json must load for the dex-face tests");
            foreach (var d in catalog.Dex) face.Add(d.id);
            return face;
        }
        [Test]
```

```
public void SecondLineageUnlock_Derives_Off_CfgDexFace()
{
    var face = CatalogDexFace();
    Assert.AreEqual(12, face.Count, "the main dex face carries the 12 cfg slots");
    Assert.IsFalse(MergeSim.SecondLineageUnlockedOf(new HashSet&lt;string&gt;(), face),
        "an empty discovered set never unlocks");
    Assert.IsFalse(MergeSim.SecondLineageUnlockedOf(null, face),
        "a null discovered set is a safe no-unlock");
    Assert.IsFalse(MergeSim.SecondLineageUnlockedOf(new HashSet&lt;string&gt;(face), null),
        "a null dex face is a safe no-unlock");
    Assert.IsFalse(MergeSim.SecondLineageUnlockedOf(new HashSet&lt;string&gt;(face), new List&lt;string&gt;()),
        "an empty dex face never unlocks (slots &gt; 0 guard)");
    var partial = new HashSet&lt;string&gt;(face);
    partial.Remove("h12");
    Assert.IsFalse(MergeSim.SecondLineageUnlockedOf(partial, face),
        "11/12 main dex discovered keeps the second lineage locked");
    var full = new HashSet&lt;string&gt;(face);
    Assert.IsTrue(MergeSim.SecondLineageUnlockedOf(full, face),
        "12/12 main dex discovered unlocks the second lineage");
    full.Add("s01");
    Assert.IsTrue(MergeSim.SecondLineageUnlockedOf(full, face),
        "extra discovered ids beyond the face do not break the unlock");
}
[Test]
public void SecondLineageFold_LatchesUnlock_And_CountOnlyUp()
{
    var sim = new MergeSim(42);
    Assert.AreEqual(0, sim.Records.SecondLineageUnlocked, "fresh records read the unlock as 0");
    Assert.AreEqual(0, sim.Records.SecondLineageAlbumCount, "fresh records read the album count as 0");
    sim.FoldSecondLineage(false, 0);
    Assert.AreEqual(0, sim.Records.SecondLineageUnlocked,
        "a false fold keeps the honest idle state until the W1-b engine lands");
    Assert.AreEqual(0, sim.Records.SecondLineageAlbumCount,
        "a zero-count fold keeps the honest idle album state");
    sim.FoldSecondLineage(true, 2);
    Assert.AreEqual(1, sim.Records.SecondLineageUnlocked, "the unlock latch flips on");
    Assert.AreEqual(2, sim.Records.SecondLineageAlbumCount, "the fold lands the album count");
    sim.FoldSecondLineage(false, 1);
    Assert.AreEqual(1, sim.Records.SecondLineageUnlocked,
        "the unlock is a one-way latch - it never rolls back");
    Assert.AreEqual(2, sim.Records.SecondLineageAlbumCount,
        "a weaker fold never lowers the album count");
    sim.FoldSecondLineage(false, 5);
    Assert.AreEqual(5, sim.Records.SecondLineageAlbumCount,
        "a stronger fold raises the album count (only-up)");
}
[Test]
public void SecondLineageRecord_Survives_SoftRestart_And_RoundTrip()
{
    var sim = new MergeSim(42);
```

```
        sim.FoldSecondLineage(true, 3);
        sim.SoftRestart();
        Assert.AreEqual(1, sim.Records.SecondLineageUnlocked,
            "soft restart keeps the unlock latch (records family survives)");
        Assert.AreEqual(3, sim.Records.SecondLineageAlbumCount,
            "soft restart keeps the album count");
        var state = sim.CaptureBoard();
        var twin = MergeSim.FromBoardState(state);
        Assert.AreEqual(1, twin.Records.SecondLineageUnlocked,
            "the unlock latch rides the board round-trip");
        Assert.AreEqual(3, twin.Records.SecondLineageAlbumCount,
            "the album count rides the board round-trip");
    }
    [Test]
    public void SecondLineageSettlement_Composes_Derivation_With_Fold()
    {
        // The settlement face composes the two W1-a APIs: derive off the
        // discovered set + cfg dex face, then fold. This composition is the
        // contract the W1-b lighting engine will drive.
        var face = CatalogDexFace();
        var sim = new MergeSim(42);
        var partial = new HashSet&lt;string&gt;(face);
        partial.Remove(face[0]);
        sim.FoldSecondLineage(MergeSim.SecondLineageUnlockedOf(partial, face), 0);
        Assert.AreEqual(0, sim.Records.SecondLineageUnlocked,
            "a partial dex keeps the fold honest");
        var full = new HashSet&lt;string&gt;(face);
        sim.FoldSecondLineage(MergeSim.SecondLineageUnlockedOf(full, face), 0);
        Assert.AreEqual(1, sim.Records.SecondLineageUnlocked,
            "the full-dex derivation folds the unlock latch on");
    }
    // U153-D04 W1-b1 (taskbook section-5 anchor): the catalog second domain is
    // the 12-slot (baseSpecies, baseTier) selection table (ids s01-s12, one slot
    // per base identity, spec-frozen count, zero invented numbers - the mapping
    // is structural: the canonical tierItems order p0..p5/c0..c5).
    [Test]
    public void SecondDex_Domain_WellFormed_And_RuleRoundTrips()
    {
        var catalog = G03GameCatalog.Load();
        Assert.NotNull(catalog, "cfg_g03_catalog.json must load for the second-domain tests");
        Assert.AreEqual(12, catalog.SecondDex.Length, "the second domain carries the 12 spec slots (s01-s12)");
        Assert.IsEmpty(catalog.Validate(), "the catalog incl. the second domain validates clean");
        var ids = new HashSet&lt;string&gt;();
        foreach (var d in catalog.SecondDex)
        {
            Assert.IsTrue(ids.Add(d.id), "second-domain ids must be unique: " + d.id);
            Assert.AreSame(d, catalog.SecondRuleOf(d.baseSpecies, d.baseTier),
                "each (baseSpecies, baseTier) key must select its own slot");
        }
        Assert.IsNull(catalog.SecondRuleOf(2, 3), "an unknown species key selects no slot");
```

```
    Assert.IsNull(catalog.SecondRuleOf(0, MergeSim.TopTier + 1), "an out-of-range tier key selects no slot");
}
// U153-D04 W1-b1: the sim branch is unlock-gated (Records.SecondLineageUnlocked
// latch) and fuses a hybrid with a base of ANY tier - the existing 150-test
// face is byte-identical pre-unlock, post-unlock the pair collapses into one
// hybrid at TopTier and the base identity rides the second_hybrid event.
[Test]
public void SecondFusion_UnlockGated_And_FusesHybridWithAnyBaseTier()
{
    var locked = new MergeSim(42);
    DropHybridOnTop(locked, 0);
    locked.Drop(0, 2, MergeSpecies.Produce);
    Assert.AreEqual(0, locked.SecondFusionCount, "pre-unlock the fusion branch stays closed");
    Assert.AreEqual(2, locked.CaptureBoard().Columns[0].Count,
        "pre-unlock the hybrid and the base both stay on the board");
    var open = new MergeSim(42);
    open.FoldSecondLineage(true, 0);
    DropHybridOnTop(open, 0);
    open.Drop(0, 2, MergeSpecies.Produce);
    Assert.AreEqual(1, open.SecondFusionCount, "the unlock latch opens the hybrid x base fusion");
    Assert.AreEqual(1, open.CaptureBoard().Columns[0].Count,
        "the pair collapses into a single hybrid item");
    var fused = open.CaptureBoard().Columns[0][0];
    Assert.AreEqual(MergeSpecies.Hybrid, fused.Species, "the fusion result keeps the Species enum untouched (hyb
    Assert.AreEqual(MergeSim.TopTier, fused.Tier, "the fusion result sits at TopTier like the hybrid branch");
    open.Drop(0, 4, MergeSpecies.Creature);
    Assert.AreEqual(2, open.SecondFusionCount, "the next hybrid x base contact fuses too (engine keeps running)"
}
// U153-D04 W1-b2: the presenter lights the second-lineage slot the fusion beat
// names (SecondRuleOf off the fused base identity) and settles the fold - first
// light only; repeat fusions of the same base identity are old news. The 12th
// main-dex adoption latches the unlock through the adoption fold, so the engine
// opens mid-run without waiting for a drain beat.
[Test]
public void PresenterLightsSecondSlot_FirstOnly_AndFoldsCount()
{
    var catalog = G03GameCatalog.Load();
    Assert.NotNull(catalog, "cfg_g03_catalog.json must load for the W1-b2 tests");
    var presenter = new G03BoardPresenter(42, catalog);
    presenter.DropFixed(0, MergeSim.TopTier, MergeSpecies.Produce);
    presenter.DropFixed(0, MergeSim.TopTier, MergeSpecies.Creature);
    Assert.AreEqual(0, presenter.Sim.SecondFusionCount,
        "pre-unlock the presenter path keeps the fusion branch closed");
    Assert.AreEqual(0, presenter.Sim.Records.SecondLineageUnlocked,
        "pre-unlock the fold is a byte-identical no-op");
    foreach (var d in catalog.Dex) presenter.AdoptDex(d.id);
    Assert.AreEqual(1, presenter.Sim.Records.SecondLineageUnlocked,
        "the 12th main-dex adoption latches the unlock through the fold");
    DropHybridOnTop(presenter.Sim, 1);
    var beats = presenter.DropFixed(1, 2, MergeSpecies.Produce);
```

```
        G03ViewCommand second = null;
        foreach (var b in beats) if (b.Kind == "second") second = b;
        Assert.NotNull(second, "the unlocked fusion surfaces a second-lineage beat");
        var slot = catalog.SecondRuleOf((int)MergeSpecies.Produce, 2);
        Assert.NotNull(slot, "the fused base identity must map to a catalog slot");
        Assert.AreEqual(slot.id, second.DexId, "the beat carries the slot the base identity lights");
        Assert.AreEqual((int)MergeSpecies.Produce, second.Species, "the beat carries the fused base species");
        Assert.AreEqual(2, second.Tier, "the beat carries the fused base tier");
        Assert.IsTrue(second.NewDex, "the first fusion of this base identity is a first light");
        Assert.AreEqual(1, presenter.DiscoveredSecondCount, "the presenter lit exactly one slot");
        Assert.IsTrue(presenter.HasSecondDex(slot.id), "the lit slot reads back through the lookup");
        Assert.AreEqual(1, presenter.Sim.Records.SecondLineageAlbumCount,
            "the drain fold settles the album count (the F3 count &gt; 0 face)");
        var repeat = presenter.DropFixed(1, 2, MergeSpecies.Produce);
        G03ViewCommand secondRepeat = null;
        foreach (var b in repeat) if (b.Kind == "second") secondRepeat = b;
        Assert.NotNull(secondRepeat, "the engine keeps fusing post-unlock");
        Assert.AreEqual(slot.id, secondRepeat.DexId, "the repeat fusion names the same slot");
        Assert.IsFalse(secondRepeat.NewDex, "a repeat fusion of the same base identity is not a new light");
        Assert.AreEqual(1, presenter.DiscoveredSecondCount, "the discovered set dedupes by slot");
        Assert.AreEqual(2, presenter.Sim.SecondFusionCount, "the engine itself keeps counting fusions");
    }
    // U153-D04 W1-b2: the second-lineage lights ride the lifetime profile slot
    // (dex2 list) - old v1 payloads without the key hydrate as an empty list.
    [Test]
    public void SecondDexLights_RoundTrip_ThroughProfileSlot()
    {
        var save = new SaveService();
        var model = new G03SaveModel();
        model.discoveredSecond = new List&lt;string&gt; { "s01", "s07" };
        G03SaveSystem.Persist(save, model, SecondSaveSlot);
        var loaded = new G03SaveModel();
        var status = G03SaveSystem.LoadInto(save, loaded, SecondSaveSlot);
        Assert.AreEqual(G03LoadStatus.Loaded, status);
        Assert.AreEqual(new List&lt;string&gt; { "s01", "s07" }, loaded.discoveredSecond,
            "the second-lineage lights survive the profile round-trip");
        Assert.AreEqual(0, loaded.discoveredDex.Count,
            "the main dex face stays independent of the second list");
        // Legacy face: a payload without the dex2 key hydrates as an empty list.
        var legacy = new Dictionary&lt;string, object&gt;
        {
            { G03SaveSystem.VersionKey, (long)G03SaveSystem.CurrentVersion },
            { "dex", new List&lt;object&gt; { "h01" } },
        };
        save.Save(SecondSaveSlot, MiniJson.Serialize(legacy));
        var migrated = new G03SaveModel();
        var legacyStatus = G03SaveSystem.LoadInto(save, migrated, SecondSaveSlot);
        Assert.AreEqual(G03LoadStatus.Loaded, legacyStatus);
        Assert.AreEqual(0, migrated.discoveredSecond.Count,
            "a payload without dex2 hydrates an empty second list (honest default)");
```

```
}
// U153-D04 W1-b2: the returning-session contract - hydration through the
// presenter adoption path re-latches the unlock and re-settles the album
// count, so a fresh session resumes the second-lineage engine from the
// profile alone (the F3 count &gt; 0 face survives the session boundary).
[Test]
public void SecondDiscovery_Survives_BootRehydration_Refold()
{
    var catalog = G03GameCatalog.Load();
    Assert.NotNull(catalog, "cfg_g03_catalog.json must load for the W1-b2 tests");
    var presenter = new G03BoardPresenter(42, catalog);
    foreach (var d in catalog.Dex) presenter.AdoptDex(d.id);
    Assert.IsTrue(presenter.AdoptSecondDex("s01"));
    Assert.IsTrue(presenter.AdoptSecondDex("s07"));
    Assert.AreEqual(1, presenter.Sim.Records.SecondLineageUnlocked);
    Assert.AreEqual(2, presenter.Sim.Records.SecondLineageAlbumCount);
    var save = new SaveService();
    var model = new G03SaveModel();
    foreach (var id in presenter.DiscoveredDex) model.discoveredDex.Add(id);
    foreach (var id in presenter.DiscoveredSecond) model.discoveredSecond.Add(id);
    G03SaveSystem.Persist(save, model, SecondSaveSlot);
    var back = new G03SaveModel();
    Assert.AreEqual(G03LoadStatus.Loaded, G03SaveSystem.LoadInto(save, back, SecondSaveSlot));
    var fresh = new G03BoardPresenter(7, catalog);
    foreach (var id in back.discoveredDex) fresh.AdoptDex(id);
    foreach (var id in back.discoveredSecond) fresh.AdoptSecondDex(id);
    Assert.AreEqual(1, fresh.Sim.Records.SecondLineageUnlocked,
        "hydration re-latches the unlock for the returning session");
    Assert.AreEqual(2, fresh.Sim.Records.SecondLineageAlbumCount,
        "hydration re-settles the album count (only-up face intact)");
    Assert.IsTrue(fresh.HasSecondDex("s01"));
    Assert.IsTrue(fresh.HasSecondDex("s07"));
    Assert.IsFalse(fresh.HasSecondDex("s03"), "unlit slots stay unlit");
    Assert.IsFalse(fresh.AdoptSecondDex("s01"), "re-adopting a hydrated light is not a first light");
}
static void DropHybridOnTop(MergeSim sim, int col)
{
    sim.Drop(col, MergeSim.TopTier, MergeSpecies.Produce);
    sim.Drop(col, MergeSim.TopTier, MergeSpecies.Creature);
    Assert.AreEqual(1, sim.CaptureBoard().Columns[col].Count,
        "helper: the TopTier cross pair hatches one hybrid");
    Assert.AreEqual(MergeSpecies.Hybrid, sim.CaptureBoard().Columns[col][0].Species,
        "helper: the column top is the hybrid");
}
// U153-D04 W1-c: every cfg second-domain key plus the album-page ui seats
// must resolve in the i18n table - a missing key falls back to the raw key
// string, which would ship untranslated text on the album cards and the
// share badge (the coverage face the album page depends on).
[Test]
public void SecondAlbum_I18n_Covers_Every_CfgSecondKey_And_UiSeat()
```

```
{
    var catalog = G03GameCatalog.Load();
    Assert.NotNull(catalog, "cfg_g03_catalog.json must load for the W1-c i18n coverage test");
    G03Text.Load();
    foreach (var d in catalog.SecondDex)
    {
        Assert.AreNotEqual(d.nameKey, G03Text.Get(d.nameKey),
            "second name key must resolve to real text: " + d.nameKey);
        Assert.AreNotEqual(d.quipKey, G03Text.Get(d.quipKey),
            "second quip key must resolve to real text: " + d.quipKey);
        Assert.IsNotEmpty(G03Text.Get(d.nameKey),
            "second name text must not be empty: " + d.nameKey);
        Assert.IsNotEmpty(G03Text.Get(d.quipKey),
            "second quip text must not be empty: " + d.quipKey);
    }
    string[] seats =
    {
        "ui.dex.hall.title.second", "ui.dex.hall.page.gosecond", "ui.dex.hall.page.gomain",
        "ui.dex.hall.hybrid", "ui.dex.hall.second.locked", "ui.dex.hall.second.hint",
        "ui.dex.hall.second.full", "ui.share.badge.second",
    };
    foreach (var key in seats)
        Assert.AreNotEqual(key, G03Text.Get(key), "album ui seat must resolve: " + key);
}
// U153-D04 W1-c: the locked second-card tease reads the hybrid term plus the
// base tier-line name (collection fuel - the player knows what to fuse), and
// the null/fallback faces stay honest.
[Test]
public void SecondLineageHint_Reads_HybridTerm_And_BaseName()
{
    var catalog = G03GameCatalog.Load();
    Assert.NotNull(catalog, "cfg_g03_catalog.json must load for the W1-c hint test");
    G03Text.Load();
    var s01 = catalog.SecondRuleOf(0, 0);
    Assert.NotNull(s01, "the (produce, 0) base identity maps to a slot");
    StringAssert.Contains(G03Text.Get("ui.dex.hall.hybrid"), G03MainView.SecondLineageHint(s01, catalog),
        "the tease names the fusion partner (the hybrid term)");
    StringAssert.Contains(G03Text.Get("g03.item.p0"), G03MainView.SecondLineageHint(s01, catalog),
        "the tease names the base item (produce tier 0)");
    var s07 = catalog.SecondRuleOf(1, 0);
    Assert.NotNull(s07, "the (creature, 0) base identity maps to a slot");
    StringAssert.Contains(G03Text.Get("g03.item.c0"), G03MainView.SecondLineageHint(s07, catalog),
        "the tease names the base item (creature tier 0)");
    Assert.AreEqual(G03Text.Get("ui.dex.hall.lineage"), G03MainView.SecondLineageHint(null, catalog),
        "a null entry falls back to the lineage label");
    Assert.AreEqual(G03Text.Get("ui.dex.hall.lineage"), G03MainView.SecondLineageHint(s01, null),
        "a null catalog falls back to the lineage label");
    var badItem = new SecondDexEntryData { id = "sXX", baseSpecies = 2, baseTier = 0 };
    Assert.AreEqual(G03Text.Get("ui.dex.hall.lineage"), G03MainView.SecondLineageHint(badItem, catalog),
        "an unknown base item falls back to the lineage label");
```

```
        }
    }
}
// G03ShareCardTests.cs
using System.Collections.Generic;
using System.IO;
using NUnit.Framework;
using Sy.G03;
using Sy.Platform;
using UnityEngine;
namespace Sy.G03.Tests
{
    // S3-T003 acceptance: RT pipeline tests (pixel-readback canary, the
    // "declared != fact" defense), the mock share full chain, and the
    // brag-moment trigger tests. The canary samples known-solid element centers
    // through the same layout solver the composer uses, plus one known seeded
    // board cell, so a blank or clipped card fails loudly even though size and
    // byte budget would have passed (F4 lesson / RT three laws #3).
    public class G03ShareCardTests
    {
        G03ShareCardComposer.ComposeResult _result;
        static G03ShareCardInfo SampleInfo(bool withCatMelon = true)
        {
            // Deterministic board face: full bottom row of tier 2, second row of
            // tier 5, single tier 4 at column 6. FromBoardState parity is pinned
            // by its own test; here we hand-build the info for pixel determinism.
            var tiers = new int[MergeSim.Cols][];
            for (int c = 0; c &lt; MergeSim.Cols; c++)
            {
                tiers[c] = new int[MergeSim.Rows];
                for (int r = 0; r &lt; MergeSim.Rows; r++) tiers[c][r] = -1;
                tiers[c][0] = 2;
                tiers[c][1] = 5;
            }
            tiers[6][1] = 4;
            return new G03ShareCardInfo
            {
                MergeCount = 88,
                MaxChain = 3,
                HybridCount = 4,
                DexCount = 12,
                HasCatMelon = withCatMelon,
                Seed = 42,
                Tiers = tiers,
            };
        }
        G03ShareCardComposer.ComposeResult ComposeSample()
        {
            G03Text.Load();
            int layer = G03ShareCardComposer.ResolveLayer();
```

```
    Assert.GreaterOrEqual(layer, 0, "G03ShareCard layer must exist in TagManager");
    var result = G03ShareCardComposer.Compose(SampleInfo(), layer);
    Assert.IsNotNull(result, "compose must succeed with a valid layer");
    _result = result;
    return result;
}
[TearDown]
public void TearDown()
{
    G03ShareCardComposer.Release(_result);
    _result = null;
}
[Test]
public void Layout_Maps_Reference_Center_To_Pixel_Center()
{
    var p = G03ShareCardLayout.ToPixel(Vector2.zero);
    Assert.AreEqual(G03ShareCardLayout.RtW * 0.5f, p.x, 0.001f);
    Assert.AreEqual(G03ShareCardLayout.RtH * 0.5f, p.y, 0.001f);
    var corner = G03ShareCardLayout.ToPixel(new Vector2(-540f, -960f));
    Assert.AreEqual(0f, corner.x, 0.001f);
    Assert.AreEqual(0f, corner.y, 0.001f);
}
[Test]
public void Compose_Produces_Png_With_Exact_Size()
{
    var result = ComposeSample();
    Assert.IsNotNull(result.Png);
    Assert.Greater(result.Png.Length, 0, "PNG bytes must exist");
    Assert.AreEqual(G03ShareCardLayout.RtW, result.Width);
    Assert.AreEqual(G03ShareCardLayout.RtH, result.Height);
    Assert.AreEqual(G03ShareCardLayout.RtW, result.Texture.width);
    Assert.AreEqual(G03ShareCardLayout.RtH, result.Texture.height);
}
[Test]
public void PixelReadback_Canary_Solid_Samples_Match()
{
    var result = ComposeSample();
    foreach (var sample in G03ShareCardLayout.SolidSamples())
    {
        Color got = G03ShareCardComposer.SampleAt(result, sample.Point);
        Assert.LessOrEqual(Mathf.Abs(got.r - sample.Color.r), 0.06f,
            sample.Name + " red channel drift");
        Assert.LessOrEqual(Mathf.Abs(got.g - sample.Color.g), 0.06f,
            sample.Name + " green channel drift");
        Assert.LessOrEqual(Mathf.Abs(got.b - sample.Color.b), 0.06f,
            sample.Name + " blue channel drift");
    }
}
[Test]
public void PixelReadback_Board_Cell_Matches_Tier_Palette()
```

```
{
    var result = ComposeSample();
    // Column 0 row 0 is seeded tier 2 in SampleInfo - the board face
    // itself is canaried, not just the chrome around it.
    Color got = G03ShareCardComposer.SampleAt(result, G03ShareCardLayout.CellCenter(0, 0));
    Color want = G03ShareCardLayout.TierColor(2);
    Assert.LessOrEqual(Mathf.Abs(got.r - want.r), 0.06f, "board cell red drift");
    Assert.LessOrEqual(Mathf.Abs(got.g - want.g), 0.06f, "board cell green drift");
    Assert.LessOrEqual(Mathf.Abs(got.b - want.b), 0.06f, "board cell blue drift");
    // And the tier-5 cell one row up must differ from tier 2 (palette is real).
    Color gotUp = G03ShareCardComposer.SampleAt(result, G03ShareCardLayout.CellCenter(0, 1));
    Color wantUp = G03ShareCardLayout.TierColor(5);
    Assert.LessOrEqual(Mathf.Abs(gotUp.r - wantUp.r), 0.06f, "tier5 cell red drift");
}
[Test]
public void Layer_Traversal_Covers_Every_Card_Object()
{
    var result = ComposeSample();
    int layer = G03ShareCardComposer.ResolveLayer();
    Assert.AreEqual(1 &lt;&lt; layer, result.Cam.cullingMask,
        "camera must cull to the share layer only");
    AssertAllOnLayer(result.Rig.transform, layer);
}
static void AssertAllOnLayer(Transform t, int layer)
{
    Assert.AreEqual(layer, t.gameObject.layer,
        "every card object must sit on the share layer: " + t.name);
    for (int i = 0; i &lt; t.childCount; i++) AssertAllOnLayer(t.GetChild(i), layer);
}
[Test]
public void BuildPayload_Carries_Title_Query_And_Local_Path()
{
    G03Text.Load();
    var info = SampleInfo();
    var payload = G03ShareCardComposer.BuildPayload(info, "C:/tmp/g03_share_card.png");
    StringAssert.Contains("88", payload.Title,
        "share title carries the merge count (bragging hook)");
    StringAssert.DoesNotContain("{m}", payload.Title, "template must be resolved");
    StringAssert.DoesNotContain("{c}", payload.Title, "template must be resolved");
    Assert.AreEqual("src=share", payload.Query, "zero-server launch attribution");
    Assert.AreEqual("C:/tmp/g03_share_card.png", payload.ImageUrl, "local path passes through");
}
[Test]
public void FromBoardState_Mirrors_Sim_Stats_And_Columns()
{
    var state = new MergeBoardState
    {
        Seed = 7,
        MergeCount = 42,
        MaxChainInOneCascade = 6,
```

```
            HybridCount = 2,
            Columns = new List&lt;MergeItem&gt;[MergeSim.Cols],
        };
        for (int c = 0; c &lt; MergeSim.Cols; c++) state.Columns[c] = new List&lt;MergeItem&gt;();
        state.Columns[0].Add(new MergeItem { Tier = 5 });
        state.Columns[0].Add(new MergeItem { Tier = 1 });
        state.Columns[3].Add(new MergeItem { Tier = 4 });
        var info = G03ShareCardInfo.FromBoardState(state, 9, true);
        Assert.AreEqual(7, info.Seed);
        Assert.AreEqual(42, info.MergeCount);
        Assert.AreEqual(6, info.MaxChain);
        Assert.AreEqual(2, info.HybridCount);
        Assert.AreEqual(9, info.DexCount);
        Assert.IsTrue(info.HasCatMelon);
        Assert.AreEqual(MergeSim.Cols, info.Tiers.Length);
        Assert.AreEqual(5, info.Tiers[0][0], "floor cell copies bottom-up");
        Assert.AreEqual(1, info.Tiers[0][1]);
        Assert.AreEqual(-1, info.Tiers[0][2], "empty cells are -1");
        Assert.AreEqual(4, info.Tiers[3][0]);
        Assert.AreEqual(-1, info.Tiers[6][0]);
    }
    [Test]
    public void FromBoardState_Tolerates_Null_State()
    {
        var info = G03ShareCardInfo.FromBoardState(null, 0, false);
        Assert.AreEqual(0, info.MergeCount);
        Assert.IsNull(info.Tiers);
        Assert.IsFalse(info.HasCatMelon);
    }
    // U153-D04 W1-c: the album brag badge face (share-face reuse) - the hall
    // share card carries the lit second-lineage count, lit only when positive.
    // The canary samples the badge's left margin (away from the centered label,
    // the same discipline as SolidSamples), so text glyphs never mask a missing
    // block.
    [Test]
    public void SecondBadge_PixelCanary_Lights_Only_With_SecondCount()
    {
        G03Text.Load();
        var plumbing = G03ShareCardInfo.FromBoardState(null, 9, false, 4);
        Assert.AreEqual(9, plumbing.DexCount);
        Assert.AreEqual(4, plumbing.SecondCount, "the 4-arg face carries the second-lineage count");
        Assert.AreEqual(0, G03ShareCardInfo.FromBoardState(null, 9, false).SecondCount,
            "the 3-arg face keeps the honest zero default for legacy callers");
        int layer = G03ShareCardComposer.ResolveLayer();
        Assert.GreaterOrEqual(layer, 0, "G03ShareCard layer must exist in TagManager");
        var seat = G03ShareCardLayout.SecondBadgeCenter + new Vector2(-320f, 0f);
        var lit = SampleInfo();
        lit.SecondCount = 3;
        _result = G03ShareCardComposer.Compose(lit, layer);
        Assert.IsNotNull(_result, "compose must succeed with a valid layer");
```

```
    Color badge = G03ShareCardComposer.SampleAt(_result, seat);
    Assert.LessOrEqual(Mathf.Abs(badge.r - G03ShareCardLayout.BadgeColor.r), 0.06f, "second badge red drift");
    Assert.LessOrEqual(Mathf.Abs(badge.g - G03ShareCardLayout.BadgeColor.g), 0.06f, "second badge green drift");
    Assert.LessOrEqual(Mathf.Abs(badge.b - G03ShareCardLayout.BadgeColor.b), 0.06f, "second badge blue drift");
    G03ShareCardComposer.Release(_result);
    _result = null;
    var dark = SampleInfo();
    dark.SecondCount = 0;
    var darkResult = G03ShareCardComposer.Compose(dark, layer);
    Assert.IsNotNull(darkResult, "compose must succeed with a valid layer");
    Color bg = G03ShareCardComposer.SampleAt(darkResult, seat);
    Assert.LessOrEqual(Mathf.Abs(bg.r - G03ShareCardLayout.BgColor.r), 0.06f, "no badge: seat reads the backgrou
    Assert.LessOrEqual(Mathf.Abs(bg.g - G03ShareCardLayout.BgColor.g), 0.06f, "no badge: seat reads the backgrou
    Assert.LessOrEqual(Mathf.Abs(bg.b - G03ShareCardLayout.BgColor.b), 0.06f, "no badge: seat reads the backgrou
    G03ShareCardComposer.Release(darkResult);
}
[Test]
public void MockShare_FullChain_LandsPngLocally_AndRecordsLocalUrl()
{
    var result = ComposeSample();
    string path = G03ShareCardComposer.WritePng(result.Png);
    Assert.IsNotEmpty(path, "compose must land a real local PNG path");
    Assert.IsTrue(File.Exists(path), "share PNG must exist on disk before Share is called");
    var payload = G03ShareCardComposer.BuildPayload(SampleInfo(), path);
    var mock = new MockPlatformService();
    mock.Share(payload);
    Assert.AreEqual(1, mock.Shares.Count, "mock bridge must record exactly one share");
    var sent = mock.Shares[0];
    Assert.AreEqual(path, sent.ImageUrl, "bridge must pass the local path through untouched");
    StringAssert.DoesNotContain("http", sent.ImageUrl,
        "red line 12: share image stays a local path, never a URL");
    StringAssert.Contains("88", sent.Title,
        "share title carries the merge count (bragging hook)");
    Assert.AreEqual("src=share", sent.Query, "zero-server launch attribution");
}
[Test]
public void Moments_CatMelon_Is_The_Flagship_Brag()
{
    var info = SampleInfo(withCatMelon: true);
    Assert.AreEqual(G03ShareMoment.CatMelon, G03ShareMoments.Evaluate(info));
    Assert.IsTrue(G03ShareMoments.ShouldOfferShare(G03ShareMoment.CatMelon));
}
[Test]
public void Moments_Priority_CatMelon_Over_Chain_And_Merge()
{
    var info = SampleInfo(withCatMelon: true);
    info.MaxChain = 9;
    info.MergeCount = 500;
    Assert.AreEqual(G03ShareMoment.CatMelon, G03ShareMoments.Evaluate(info),
        "the hook promise realized outranks records");
```

```
}
[Test]
public void Moments_Chain_And_Merge_Records_Trigger()
{
    var chain = SampleInfo(withCatMelon: false);
    chain.MaxChain = G03ShareMoments.ChainBragMin;
    chain.MergeCount = 3;
    Assert.AreEqual(G03ShareMoment.ChainRecord, G03ShareMoments.Evaluate(chain));
    var merge = SampleInfo(withCatMelon: false);
    merge.MaxChain = G03ShareMoments.ChainBragMin - 1;
    merge.MergeCount = G03ShareMoments.MergeBragMin;
    Assert.AreEqual(G03ShareMoment.MergeRecord, G03ShareMoments.Evaluate(merge));
}
[Test]
public void Moments_Plain_Board_Is_Not_Shareable_And_Null_Safe()
{
    var plain = SampleInfo(withCatMelon: false);
    plain.MaxChain = 1;
    plain.MergeCount = 4;
    Assert.AreEqual(G03ShareMoment.None, G03ShareMoments.Evaluate(plain));
    Assert.IsFalse(G03ShareMoments.ShouldOfferShare(G03ShareMoment.None));
    Assert.AreEqual(G03ShareMoment.None, G03ShareMoments.Evaluate(null));
}
// S4-T004: share-open attribution (the src=share consumption leg that the
// S3-T003 forward chain was missing). Mock suite injects the launch query.
sealed class CaptureSink : Sy.Telemetry.TelemetryService.ISink
{
    public readonly List&lt;string&gt; Events = new();
    public void Write(string evtId, Dictionary&lt;string, string&gt; fields)
    {
        string raw = fields != null &amp;&amp; fields.TryGetValue("raw", out var r) ? r : "";
        Events.Add(evtId + "|" + raw);
    }
}
[Test]
public void Attribution_Detects_Share_Token_Only()
{
    Assert.IsTrue(G03ShareAttribution.IsShareOpen("src=share&amp;seed=42"),
        "share token detected inside a compound query");
    Assert.IsFalse(G03ShareAttribution.IsShareOpen(null),
        "organic open (no query at all) is not a share open");
    Assert.IsFalse(G03ShareAttribution.IsShareOpen("seed=42"),
        "a seed query without the token is not a share open");
    Assert.IsFalse(G03ShareAttribution.IsShareOpen("SRC=SHARE"),
        "token match is ordinal (case-sensitive)");
}
[Test]
public void Attribution_CaptureAtBoot_Tracks_Unified_evt_share()
{
    var launch = new MockLaunchInfo { Next = new LaunchQuery { Raw = "src=share&amp;seed=42" } };
```

```
            var telemetry = new Sy.Telemetry.TelemetryService();
            var sink = new CaptureSink();
            telemetry.AddSink(sink);
            bool fired = G03ShareAttribution.CaptureAtBoot(launch, telemetry);
            Assert.IsTrue(fired, "a share-card open captures at boot");
            Assert.AreEqual(1, sink.Events.Count, "exactly one attribution event lands");
            Assert.AreEqual(Sy.Telemetry.EventIds.Share + "|src=share&amp;seed=42", sink.Events[0],
                "the unified evt_share id carries the raw launch query for attribution");
        }
        [Test]
        public void Attribution_CaptureAtBoot_Silent_For_Organic_And_Null_Faces()
        {
            var telemetry = new Sy.Telemetry.TelemetryService();
            var sink = new CaptureSink();
            telemetry.AddSink(sink);
            Assert.IsFalse(G03ShareAttribution.CaptureAtBoot(new MockLaunchInfo(), telemetry),
                "null launch query (organic open) stays silent");
            Assert.IsFalse(G03ShareAttribution.CaptureAtBoot(
                new MockLaunchInfo { Next = new LaunchQuery { Raw = "seed=42" } }, telemetry),
                "a non-share query stays silent");
            Assert.IsFalse(G03ShareAttribution.CaptureAtBoot(null, telemetry),
                "a null launch face is tolerated");
            Assert.AreEqual(0, sink.Events.Count, "no attribution event fired");
        }
    }
}
// G03TuneSweepTests.cs
using System.Collections.Generic;
using NUnit.Framework;
using Sy.G03;
namespace Sy.G03.Tests
{
    // S3-T002 slice A acceptance (taskbook: sweep harness + window-center pick as pure
    // code; the batchmode grid run itself is slice B on a RAM&gt;=30% wave). The greedy bot
    // mirrors the E4 GrayboxCheck stage-2 face verbatim, so the E4-default corner must
    // reproduce the published E4 evidence numbers byte-for-byte - that pins the harness
    // to the anchor before any tuned value can enter cfg_g03_catalog economy.
    public class G03TuneSweepTests
    {
        static TuneSweep.SweepConfig Cfg(int spawnTop, double heatCap, double rotate)
        {
            return new TuneSweep.SweepConfig { SpawnTopTier = spawnTop, HeatCap = heatCap, RotatePeriodSec = rotate };
        }
        [Test]
        public void E4_Default_Corner_Reproduces_E4_Bot_Evidence_Byte_Parity()
        {
            // E4 evidence (attempt2, 2026-09-19): 133 merges / 200 hands,
            // rotate-chain merges 11, hybrids 4, heat bursts 8, board never died.
            // C990 lock law (U153-D01 d-p4): production HeatPerMerge moved
            // 6.0 -&gt; 1.40 (d-p3c qualifying cell), so the anchor pins the
```

```
    // HARNESS at the E4-era valve passed explicitly through the
    // calibration ctor - byte-parity with the published evidence
    // survives the production value moving on record.
    // C1024 re-anchor (2026-09-24): the OOM cascade fix (resolve latch
    // + deferred burst drain) removed the decompose-pair inline re-merge
    // from the spawning pass, so this corner now plays 138 merges
    // (measured 2026-09-24 20:11 test run). 138 is the honest post-fix
    // re-derivation; the 2026-09-19 graybox evidence file stays
    // untouched as the cascade-coupled historical record.
    var e4Cfg = Cfg(5, 100, 30);
    e4Cfg.HeatPerMerge = 6.0;   // E4-era valve (pre-C990) - the anchor face
    var m = TuneSweep.RunOneBot(4242, e4Cfg, true);
    Assert.AreEqual(200, m.Drops, "E4 bot ran the full 200 hands");
    Assert.AreEqual(138, m.Merges, "E4 face: 138 merges (C1024 re-anchor)");
    Assert.AreEqual(8, m.RotateChainMerges, "E4 face: 8 rotate-chain merges (C1025 re-anchor; E4-era 11 was casc
    Assert.AreEqual(4, m.Hybrids, "E4 face: 4 hybrids");
    Assert.AreEqual(9, m.HeatBursts, "E4 face: 9 heat bursts (C1025 re-anchor; E4-era 8 was cascade-coupled)");
    Assert.AreEqual(-1, m.FirstOverflowAtDrop, "E4 face: no overflow inside 200 hands");
}
[Test]
public void E4_Default_Corner_Median_Qualifies_All_Windows()
{
    var cfg = Cfg(5, 100, 30);
    var runs = new List&lt;TuneSweep.BotMetrics&gt;();
    foreach (int seed in new[] { 11, 22, 33, 44, 55 })
        runs.Add(TuneSweep.RunOneBot(seed, cfg, true));
    // median pass is asserted through the public window predicate only; the
    // median aggregation itself is exercised by determinism + parity tests.
    bool anyQualified = false;
    foreach (var r in runs) if (TuneSweep.Qualifies(r)) anyQualified = true;
    Assert.IsTrue(anyQualified, "E4-default corner keeps every north-star window");
}
[Test]
public void RunOneBot_Is_Deterministic_And_Seed_Sensitive()
{
    var cfg = Cfg(4, 100, 30);
    var a = TuneSweep.RunOneBot(11, cfg, true);
    var b = TuneSweep.RunOneBot(11, cfg, true);
    Assert.AreEqual(a.Drops, b.Drops);
    Assert.AreEqual(a.Merges, b.Merges);
    Assert.AreEqual(a.MergesPerHand, b.MergesPerHand, 1e-12);
    Assert.AreEqual(a.Chain3s, b.Chain3s);
    Assert.AreEqual(a.Hybrids, b.Hybrids);
    Assert.AreEqual(a.RotateChainMerges, b.RotateChainMerges);
    Assert.AreEqual(a.HeatBursts, b.HeatBursts);
    Assert.AreEqual(a.Overflows, b.Overflows);
    Assert.AreEqual(a.FirstOverflowAtDrop, b.FirstOverflowAtDrop);
    Assert.AreEqual(a.EndItems, b.EndItems);
    var c = TuneSweep.RunOneBot(12, cfg, true);
    Assert.AreNotEqual(a.Merges, c.Merges, "different seed must play a different run");
```

```
        }
        [Test]
        public void PickWindowCenter_Chooses_Value_Closest_To_Center_Per_Axis()
        {
            var q = new List&lt;KeyValuePair&lt;TuneSweep.SweepConfig, TuneSweep.BotMetrics&gt;&gt;();
            q.Add(new KeyValuePair&lt;TuneSweep.SweepConfig, TuneSweep.BotMetrics&gt;(Cfg(5, 100, 30), Met(0.66, 2)));
            q.Add(new KeyValuePair&lt;TuneSweep.SweepConfig, TuneSweep.BotMetrics&gt;(Cfg(4, 100, 30), Met(0.60, 1)));
            q.Add(new KeyValuePair&lt;TuneSweep.SweepConfig, TuneSweep.BotMetrics&gt;(Cfg(3, 70, 20), Met(0.50, 1)));
            var pick = TuneSweep.PickWindowCenter(q);
            Assert.IsTrue(pick.HasValue);
            Assert.AreEqual(5, pick.Value.SpawnTopTier, "0.66 is closest to the 0.65 center");
            Assert.AreEqual(100, pick.Value.HeatCap);
            Assert.AreEqual(30, pick.Value.RotatePeriodSec);
            // Equal deviation on different axis values: fewer overflows wins.
            var q2 = new List&lt;KeyValuePair&lt;TuneSweep.SweepConfig, TuneSweep.BotMetrics&gt;&gt;();
            q2.Add(new KeyValuePair&lt;TuneSweep.SweepConfig, TuneSweep.BotMetrics&gt;(Cfg(5, 100, 30), Met(0.65, 2)));
            q2.Add(new KeyValuePair&lt;TuneSweep.SweepConfig, TuneSweep.BotMetrics&gt;(Cfg(4, 100, 30), Met(0.65, 1)));
            var pick2 = TuneSweep.PickWindowCenter(q2);
            Assert.IsTrue(pick2.HasValue);
            Assert.AreEqual(4, pick2.Value.SpawnTopTier, "tie broken by fewer overflows, then lower axis value");
        }
        [Test]
        public void PickWindowCenter_Empty_Input_Returns_Null_And_Median_Works()
        {
            Assert.IsFalse(TuneSweep.PickWindowCenter(new List&lt;KeyValuePair&lt;TuneSweep.SweepConfig, TuneSweep.BotMe
            Assert.IsFalse(TuneSweep.PickWindowCenter(null).HasValue);
            Assert.AreEqual(3.0, TuneSweep.Median(new List&lt;double&gt; { 1, 3, 5 }), 1e-12, "odd middle");
            Assert.AreEqual(3.5, TuneSweep.Median(new List&lt;double&gt; { 1, 2, 5, 6 }), 1e-12, "even average of middle
        }
        static TuneSweep.BotMetrics Met(double mergesPerHand, int overflows)
        {
            return new TuneSweep.BotMetrics
            {
                Drops = 200,
                Merges = (int)(mergesPerHand * 200),
                MergesPerHand = mergesPerHand,
                Chain3s = 1,
                Hybrids = 1,
                RotateChainMerges = 1,
                HeatBursts = 1,
                Overflows = overflows,
                FirstOverflowAtDrop = -1,
                EndItems = 10,
            };
        }
    }
}
// G03VentTests.cs
using NUnit.Framework;
using Sy.G03;
```

```
namespace Sy.G03.Tests
{
    // U153-D03 slice b acceptance (C1019 spec block: RequestVent + VentHeatRefund
    // S1 tuning window + vent/vent_deny_empty events + VentCount independent counter
    // + BoardState four-port round-trip, zero auto-vent path, heat cap keeps its D01
    // ladder value). EM four faces: vent takes effect (deterministic lowest-tier /
    // leftmost-column / topmost-occurrence pick + clamped refund) / empty board
    // denies with zero drift / passive valve counters zero drift with a real
    // (unclamped) refund / round-trip across all four ports (capture, resync,
    // revive restore, soft restart) including the legacy-snapshot default. All
    // scenarios are rng-free (no drops), so every trajectory is deterministic.
    public class G03VentTests
    {
        static int CountEvents(MergeSim sim, string kind)
        {
            int n = 0;
            foreach (var e in sim.Events) if (e.Kind == kind) n++;
            return n;
        }
        static string LastEventDetail(MergeSim sim, string kind)
        {
            string detail = null;
            foreach (var e in sim.Events) if (e.Kind == kind) detail = e.Detail;
            return detail;
        }
        [Test]
        public void Vent_SacrificesLowestTierLeftmostColumn_RefundsHeatClamped()
        {
            var sim = new MergeSim(20260924);
            sim.SeedColumn(0, new int[] { 3 }, MergeSpecies.Produce);
            sim.SeedColumn(1, new int[] { 1, 2 }, MergeSpecies.Produce);
            sim.SeedColumn(2, new int[] { 1 }, MergeSpecies.Produce);
            Assert.AreEqual(4, sim.ItemCount);
            Assert.IsTrue(sim.RequestVent(), "vent takes effect on a loaded board");
            Assert.AreEqual(1, sim.VentCount, "the active seat counts in the independent counter");
            Assert.AreEqual(0, sim.HeatBurstCount, "passive burst counter untouched (zero drift)");
            Assert.AreEqual(0, sim.BurstStrikes);
            Assert.AreEqual(3, sim.ItemCount, "exactly one item sacrificed");
            Assert.AreEqual(2, sim.StackTopTier(1), "the tier-1 in col 1 was the leftmost lowest-tier pick");
            Assert.AreEqual(3, sim.StackTopTier(0), "higher tiers are never the sacrifice");
            Assert.AreEqual(1, sim.StackTopTier(2), "col 2 keeps its tier 1: leftmost column wins the tie");
            Assert.AreEqual(1, CountEvents(sim, "vent"));
            Assert.IsTrue(LastEventDetail(sim, "vent").Contains("col=1") &amp;amp;&amp;amp; LastEventDetail(sim, "vent").Contain
                "vent payload carries the sacrifice slot: " + LastEventDetail(sim, "vent"));
            Assert.AreEqual(0.0, sim.Heat, 1e-9, "refund clamps at zero on an empty gauge");
            Assert.IsTrue(sim.RequestVent(), "second vent picks the next lowest (col 2 tier 1)");
            Assert.IsTrue(sim.RequestVent(), "third vent picks col 1 tier 2");
            Assert.IsTrue(sim.RequestVent(), "a lone tier 3 IS the lowest tier on that board");
            Assert.AreEqual(4, sim.VentCount);
            Assert.AreEqual(0, sim.ItemCount, "the drain is exhaustive and deterministic");
```

```
    Assert.IsFalse(sim.RequestVent(), "the drained board now denies");
    Assert.AreEqual(4, sim.VentCount, "a deny never counts");
    Assert.AreEqual(1, CountEvents(sim, "vent_deny_empty"));
}
[Test]
public void Vent_EmptyBoard_Denies_NothingChanges()
{
    var sim = new MergeSim(20260924);
    Assert.AreEqual(0, sim.ItemCount);
    Assert.IsFalse(sim.RequestVent(), "empty board denies the vent");
    Assert.AreEqual(1, CountEvents(sim, "vent_deny_empty"), "the deny rides its own event face");
    Assert.AreEqual(0, sim.VentCount);
    Assert.AreEqual(0, CountEvents(sim, "vent"), "no vent event on a deny");
    Assert.AreEqual(0.0, sim.Heat, 1e-9);
    Assert.AreEqual(1, sim.Events.Count, "the deny is the only event");
    sim.SeedColumn(0, new int[] { 2 }, MergeSpecies.Produce);
    Assert.IsTrue(sim.RequestVent(), "seeding then venting works");
    Assert.AreEqual(1, sim.VentCount);
    Assert.AreEqual(1, CountEvents(sim, "vent_deny_empty"), "the earlier deny is not re-emitted");
}
[Test]
public void Vent_RealRefund_ValveAndBurstCountersZeroDrift()
{
    var sim = new MergeSim(13, 30.0, 100.0, 40.0, 0.0);   // calib ctor: tier-1 merge adds 8.0 heat
    sim.SeedColumn(0, new int[] { 1, 1 }, MergeSpecies.Produce);
    sim.SeedColumn(1, new int[] { 1, 1 }, MergeSpecies.Produce);
    Assert.IsTrue(sim.RequestRotate());
    Assert.AreEqual(2, sim.MergeCount, "the seeded pairs both merge on the turn");
    Assert.AreEqual(16.0, sim.Heat, 1e-9, "deterministic heat: 2 merges x 8.0");
    Assert.IsTrue(sim.RequestVent());
    Assert.AreEqual(10.0, sim.Heat, 1e-9, "real (unclamped) refund: 16.0 - 6.0 (C1026 locked VentHeatRefund)");
    Assert.AreEqual(1, sim.VentCount);
    Assert.IsTrue(LastEventDetail(sim, "vent").Contains("heatDelta=6"),
        "the refund delta rides the payload: " + LastEventDetail(sim, "vent"));
    Assert.AreEqual(0, sim.HeatBurstCount, "venting never counts a burst");
    Assert.AreEqual(0, sim.BurstStrikes);
    Assert.AreEqual(0, CountEvents(sim, "heat_burst"));
    Assert.AreEqual(0, CountEvents(sim, "decompose"), "vent never triggers the passive decompose");
    Assert.AreEqual(0, CountEvents(sim, "burst_strike"));
    Assert.AreEqual(100.0, sim.EffectiveHeatCap, 1e-9, "cap keeps its D01 ladder value (vent never touches HeatC
    Assert.IsTrue(sim.RequestVent(), "second sacrifice drains the board");
    Assert.AreEqual(4.0, sim.Heat, 1e-9, "the 10.0 gauge steps to 4.0 on the second refund (locked 6.0)");
    Assert.IsFalse(sim.RequestVent(), "board empty again");
    Assert.AreEqual(2, sim.VentCount);
    Assert.AreEqual(4.0, sim.Heat, 1e-9, "a deny never moves the gauge");
}
[Test]
public void Vent_RoundTripFourPorts_ReviveRestoresSnapshot_SoftRestartClears()
{
    var sim = new MergeSim(11, 30.0, 1.0, 5.0, 0.0);   // calib ctor: cap 1.0, tier-1 merge adds 1.0
```

```
            sim.SeedColumn(0, new int[] { 1, 1 }, MergeSpecies.Produce);
            sim.SeedColumn(1, new int[] { 1, 1 }, MergeSpecies.Produce);
            Assert.IsTrue(sim.RequestRotate());
            Assert.AreEqual(2, sim.BurstStrikes, "both merges burst at the tight cap (warning band)");
            Assert.AreEqual(2, sim.HeatBurstCount);
            Assert.IsTrue(sim.RequestVent(), "active vent between the strikes");
            Assert.AreEqual(1, sim.VentCount);
            sim.AdvanceTo(8.05);   // manual rotate cooldown (const fallback) recovers
            Assert.IsTrue(sim.RequestRotate());
            Assert.IsTrue(sim.IsRunOver, "the third strike latches the terminal judgment");
            Assert.IsTrue(sim.HasReviveOffer, "the pre-fatal-burst snapshot is held");
            Assert.IsTrue(sim.RequestVent(), "post-terminal vent: the active seat still serves after the latch");
            Assert.AreEqual(2, sim.VentCount, "the live counter is 2 after the post-terminal vent");
            Assert.AreEqual(1, CountEvents(sim, "run_over"));
            sim.AcceptRevive();
            Assert.AreEqual(1, sim.VentCount, "the revive restore port replays the SNAPSHOTTED count (not the live 2)");
            Assert.IsFalse(sim.IsRunOver, "precise recovery un-latches the terminal state");
            var st = sim.CaptureBoard();
            Assert.AreEqual(1, st.VentCount, "the capture port carries the active seat");
            var restored = MergeSim.FromBoardState(st);
            Assert.AreEqual(1, restored.VentCount, "the resync port carries the active seat");
            var legacy = new MergeBoardState { Seed = 5 };   // pre-D03 snapshot: field defaults to 0
            Assert.AreEqual(0, MergeSim.FromBoardState(legacy).VentCount, "legacy snapshots default to zero");
            restored.SoftRestart();
            Assert.AreEqual(0, restored.VentCount, "fresh run clears the active vent seat");
        }
    }
}
// G03ViewTests.cs
using NUnit.Framework;
using Sy.G03;
using UnityEngine;
namespace Sy.G03.Tests
{
    // S1-T003 view acceptance: the presenter must mirror sim events 1:1 as typed view
    // commands (fall/flash/chain/hybrid/rotate/overflow), the board layout solver must pin
    // the grid math, and the art fallback chain must stay procedural-first until real art
    // lands (X027 view rule). Pure EditMode - the MonoBehaviour host renders only.
    public class G03ViewTests
    {
        [Test]
        public void BoardLayout_Pins_Grid_Math_And_Column_Hit_Test()
        {
            Assert.AreEqual(new Vector2(-420f, -690f), G03BoardLayout.CellCenter(0, 0), "floor-left cell");
            Assert.AreEqual(new Vector2(420f, 570f), G03BoardLayout.CellCenter(6, 9), "top-right cell");
            Assert.AreEqual(new Vector2(0f, -130f), G03BoardLayout.CellCenter(3, 4), "board center column");
            Assert.AreEqual(0, G03BoardLayout.ColumnAt(-489f), "leftmost pixel maps col 0");
            Assert.AreEqual(3, G03BoardLayout.ColumnAt(-1f), "center pixel maps col 3");
            Assert.AreEqual(6, G03BoardLayout.ColumnAt(489f), "rightmost pixel maps col 6");
            Assert.AreEqual(-1, G03BoardLayout.ColumnAt(-491f), "outside left is refused");
```

```
    Assert.AreEqual(-1, G03BoardLayout.ColumnAt(491f), "outside right is refused");
    Assert.AreEqual(420f, G03BoardLayout.SpawnPoint(6).x, "spawn shares the column x");
    Assert.Greater(G03BoardLayout.SpawnPoint(0).y, G03BoardLayout.CellCenter(0, 9).y,
        "spawn point sits above the rim");
}
[Test]
public void Presenter_Commands_Mirror_Sim_Events_One_To_One()
{
    var presenter = new G03BoardPresenter(7, G03GameCatalog.Load());
    Assert.IsNotNull(presenter, "catalog must load in batchmode");
    int commands = 0;
    commands += presenter.DropFixed(0, 1, MergeSpecies.Produce).Length;
    commands += presenter.DropFixed(0, 1, MergeSpecies.Produce).Length; // merge -&gt; tier 2
    commands += presenter.Advance(31.0).Length;                         // auto rotate at 30s
    Assert.AreEqual(presenter.Sim.Events.Count, commands,
        "every sim event drains into exactly one view command, in order");
}
[Test]
public void Presenter_MergeFlash_And_Chain_Bounce_From_Cascade()
{
    var presenter = new G03BoardPresenter(11, G03GameCatalog.Load());
    Assert.IsNotNull(presenter);
    presenter.Sim.SeedColumn(2, new[] { 2, 2, 1 }, MergeSpecies.Produce);
    var commands = presenter.DropFixed(2, 1, MergeSpecies.Produce);
    int flashes = 0, chains = 0, chainCount = 0, falls = 0;
    foreach (var cmd in commands)
    {
        if (cmd.Kind == "flash") flashes++;
        if (cmd.Kind == "chain") { chains++; chainCount = cmd.Count; }
        if (cmd.Kind == "fall") falls++;
    }
    Assert.AreEqual(1, falls, "one fall beat for the dropped item");
    Assert.AreEqual(2, flashes, "two merges in the cascade");
    Assert.AreEqual(1, chains, "one chain bounce for the whole cascade");
    Assert.AreEqual(2, chainCount, "chain bounce carries the cascade merge count");
    Assert.AreEqual(3, presenter.Sim.StackTopTier(2), "column top resolves to tier 3");
    Assert.AreEqual(2, presenter.Sim.MaxChainInOneCascade, "chain depth 2 recorded");
}
[Test]
public void Presenter_Hybrid_Command_Follows_Graybox_Parity()
{
    var presenter = new G03BoardPresenter(13, G03GameCatalog.Load());
    Assert.IsNotNull(presenter);
    presenter.Sim.SeedColumn(4, new[] { MergeSim.TopTier }, MergeSpecies.Produce);
    var commands = presenter.DropFixed(4, MergeSim.TopTier, MergeSpecies.Creature);
    bool hybrid = false;
    foreach (var cmd in commands) if (cmd.Kind == "hybrid") hybrid = true;
    Assert.IsTrue(hybrid, "cross-type TopTier contact must emit the hybrid command");
    Assert.AreEqual(1, presenter.Sim.HybridCount, "one cat-melon hatched");
    var top = presenter.Sim.ColumnItems(4);
```

```
    Assert.AreEqual(1, top.Count, "the pair collapses into the single hybrid item");
    Assert.AreEqual(MergeSim.TopTier, top[0].Tier, "hybrid sits at the top tier");
    Assert.AreEqual(MergeSpecies.Hybrid, top[0].Species, "hybrid species flagged");
}
[Test]
public void Presenter_Rotate_Emits_Command_And_Resyncs_Board()
{
    var presenter = new G03BoardPresenter(17, G03GameCatalog.Load());
    Assert.IsNotNull(presenter);
    presenter.Sim.SeedColumn(0, new[] { 3, 2, 1, 1 }, MergeSpecies.Produce);
    presenter.Sim.SeedColumn(1, new[] { 1, 1 }, MergeSpecies.Produce);
    presenter.Sim.SeedColumn(4, new[] { 1, 1 }, MergeSpecies.Produce);
    var commands = presenter.Advance(31.0);
    int rotates = 0;
    foreach (var cmd in commands) if (cmd.Kind == "rotate") rotates++;
    Assert.AreEqual(1, rotates, "one rotate beat at the 30s cadence");
    Assert.AreEqual(1, presenter.Sim.RotateCount, "sim rotated exactly once");
    int total = 0;
    for (int c = 0; c &lt; MergeSim.Cols; c++) total += presenter.Sim.ColumnItems(c).Count;
    Assert.AreEqual(presenter.Sim.ItemCount, total,
        "ColumnItems read access must cover the whole rotated board (resync oracle)");
}
[Test]
public void Presenter_Overflow_Drop_Refused_With_Command()
{
    var presenter = new G03BoardPresenter(19, G03GameCatalog.Load());
    Assert.IsNotNull(presenter);
    var fill = new int[MergeSim.Rows];
    presenter.Sim.SeedColumn(0, fill, MergeSpecies.Produce);
    var commands = presenter.DropFixed(0, 1, MergeSpecies.Produce);
    bool overflow = false;
    foreach (var cmd in commands) if (cmd.Kind == "overflow") overflow = true;
    Assert.IsTrue(overflow, "refused drop on a full column must emit the overflow command");
    Assert.AreEqual(1, presenter.Sim.OverflowCount, "overflow pressure stat counted");
    Assert.AreEqual(MergeSim.Rows, presenter.Sim.ColumnItems(0).Count, "column not overfilled");
}
[Test]
public void ArtFallback_Stem_Convention_And_Catalog_Tints()
{
    var catalog = G03GameCatalog.Load();
    Assert.IsNotNull(catalog);
    var presenter = new G03BoardPresenter(23, catalog);
    Assert.AreEqual("g03_p2", G03BoardPresenter.ArtStem(0, 2, catalog),
        "art pipeline stem convention: g03_&lt;tierItem id&gt;");
    Assert.AreEqual("g03_c5", G03BoardPresenter.ArtStem(1, 5, catalog),
        "creature line stem resolves too");
    Color expect;
    Assert.IsTrue(ColorUtility.TryParseHtmlString(catalog.TierItem(1, 5).colorHex, out expect),
        "catalog tint must parse");
    Assert.AreEqual(expect, presenter.ItemColor(1, 5),
```

```
            "fallback tint equals the catalog jelly color (procedural placeholder)");
        Assert.IsFalse(string.IsNullOrEmpty(presenter.ItemName(1, 5)),
            "i18n name resolves for the board label");
    }
    [Test]
    public void Presenter_Heat_Gauge_Fraction_And_Warn_Ramp()
    {
        var presenter = new G03BoardPresenter(29, G03GameCatalog.Load());
        Assert.IsNotNull(presenter);
        Assert.AreEqual(0.0, presenter.HeatFraction(), 1e-9, "fresh board heat starts empty");
        Assert.IsFalse(presenter.HeatWarn(), "no warn at zero heat");
        // C990 lock (HeatPerMerge 6.0 -&gt; 1.40, U153-D01 d-p4): the scripted
        // stack scales to keep the gauge in the warn band - nine seeded
        // TopTier items + one drop resolve nine cap merges per column,
        // 7 columns x 9 x 1.40 = 88.2 heat (&gt;= the 80% warn line, under
        // the 100 burst cap, so this test stays on the gauge face).
        int[] capStack = new int[9];
        for (int i = 0; i &lt; capStack.Length; i++) capStack[i] = MergeSim.TopTier;
        for (int c = 0; c &lt; MergeSim.Cols; c++)
            presenter.Sim.SeedColumn(c, capStack, MergeSpecies.Produce);
        for (int c = 0; c &lt; MergeSim.Cols; c++)
            presenter.DropFixed(c, MergeSim.TopTier, MergeSpecies.Produce);
        Assert.AreEqual(88.2, presenter.Sim.Heat, 1e-9, "7 columns x 9 cap merges x 1.40 = 88.2");
        Assert.AreEqual(0.882, presenter.HeatFraction(), 1e-9, "fraction mirrors the gauge");
        Assert.IsTrue(presenter.HeatWarn(), "risk banner flips on at &gt;= 80% of the cap");
    }
    [Test]
    public void Presenter_Rotate_Countdown_Remaining_Window()
    {
        var presenter = new G03BoardPresenter(31, G03GameCatalog.Load());
        Assert.IsNotNull(presenter);
        presenter.Advance(10.0);
        Assert.AreEqual(20.0, presenter.RotateRemainingSec(), MergeSim.TickSec + 1e-6,
            "10s in -&gt; 20s to the turn (tick-granularity tolerance)");
        presenter.Advance(25.0);
        Assert.AreEqual(5.0, presenter.RotateRemainingSec(), MergeSim.TickSec + 1e-6,
            "25s in -&gt; 5s urgent window (tick-granularity tolerance)");
        presenter.Advance(31.0);   // absolute virtual time 31s: the turn fired at 30s
        Assert.AreEqual(1, presenter.Sim.RotateCount, "one board turn on the 30s cadence");
        Assert.AreEqual(29.0, presenter.RotateRemainingSec(), MergeSim.TickSec + 1e-6,
            "the countdown resets right after the turn");
    }
    [Test]
    public void Presenter_Hybrid_Quip_Resolves_Top_Dex_Roast_Line()
    {
        var presenter = new G03BoardPresenter(37, G03GameCatalog.Load());
        Assert.IsNotNull(presenter);
        presenter.Sim.SeedColumn(4, new[] { MergeSim.TopTier }, MergeSpecies.Produce);
        var commands = presenter.DropFixed(4, MergeSim.TopTier, MergeSpecies.Creature);
        bool laugh = false;
```

```
    foreach (var cmd in commands) if (cmd.Kind == "laugh") laugh = true;
    Assert.IsTrue(laugh, "the hatch carries the dedicated laugh command for the quip barrage");
    Assert.IsFalse(string.IsNullOrEmpty(presenter.HybridQuip()), "quip must resolve");
    Assert.AreEqual(G03Text.Get("g03.dex.h12.quip"), presenter.HybridQuip(),
        "quip resolves the h12 roast line through the catalog lineage rule");
}
[Test]
public void Presenter_Kind_Mapping_Surfaces_Distinct_Mechanism_Beats()
{
    var presenter = new G03BoardPresenter(41, G03GameCatalog.Load());
    Assert.IsNotNull(presenter);
    presenter.Sim.EmitExternal("rotate_chain", "merges=5");
    presenter.Sim.EmitExternal("laugh_signal", "cat-melon caption");
    presenter.Sim.EmitExternal("decompose", "col=3 fromTier=4");
    presenter.Sim.EmitExternal("heat_burst", "forced decompose of most valuable stack");
    var commands = presenter.Advance(0.2);
    int chainBursts = 0, laughs = 0, decomposes = 0, heats = 0, chainCount = 0, decomposeCol = -1;
    foreach (var cmd in commands)
    {
        if (cmd.Kind == "chain") { chainBursts++; chainCount = cmd.Count; }
        if (cmd.Kind == "laugh") laughs++;
        if (cmd.Kind == "decompose") { decomposes++; decomposeCol = cmd.Col; }
        if (cmd.Kind == "heat") heats++;
    }
    Assert.AreEqual(1, chainBursts, "the turn cascade surfaces as a chain burst");
    Assert.AreEqual(5, chainCount, "burst carries its merge count");
    Assert.AreEqual(1, laughs, "laugh beat is its own command kind");
    Assert.AreEqual(1, decomposes, "decomposition is its own command kind");
    Assert.AreEqual(3, decomposeCol, "decompose command parses the hit column");
    Assert.AreEqual(1, heats, "the valve burst is its own command kind");
}
[Test]
public void Presenter_Heat_Burst_Beats_And_Gauge_Reset()
{
    var presenter = new G03BoardPresenter(43, G03GameCatalog.Load());
    Assert.IsNotNull(presenter);
    for (int c = 0; c &lt; MergeSim.Cols; c++)
        presenter.Sim.SeedColumn(c, new[] { MergeSim.TopTier, MergeSim.TopTier }, MergeSpecies.Produce);
    bool burst = false, decompose = false;
    // C990 lock (HeatPerMerge 6.0 -&gt; 1.40): the valve needs more scripted
    // drops to cross the cap - round 1 resolves two cap merges per
    // column, rounds 2+ one merge per column (deterministic fixed
    // point: every column rests at a single cap item between rounds),
    // so heat goes 19.6 then +9.8 per round and crosses 100 inside
    // round 10; the bound 12 leaves margin.
    for (int round = 0; round &lt; 12 &amp;&amp; !burst; round++)
    {
        for (int c = 0; c &lt; MergeSim.Cols &amp;&amp; !burst; c++)
        {
            foreach (var cmd in presenter.DropFixed(c, MergeSim.TopTier, MergeSpecies.Produce))
```

```
                    {
                        if (cmd.Kind == "heat") burst = true;
                        if (cmd.Kind == "decompose") decompose = true;
                    }
                }
            }
            Assert.IsTrue(burst, "the valve blows once the gauge crosses the cap");
            Assert.IsTrue(decompose, "the burst forces the decomposition command");
            Assert.GreaterOrEqual(presenter.Sim.HeatBurstCount, 1, "burst stat recorded");
            Assert.AreEqual(40.0, presenter.Sim.Heat, 1e-9,
                "the valve vents the gauge back to the 40% reset line");
            Assert.AreEqual(0.4, presenter.HeatFraction(), 1e-9, "fraction follows the reset");
        }
    }
}
// MergeSimParityTests.cs
using System.Collections.Generic;
using System.Diagnostics;
using System.Text;
using NUnit.Framework;
using Sy.G03;
namespace Sy.G03.Tests
{
    // S1-T001 parity anchors: every threshold mirrors the E4 attempt2 gate
    // (Design/evidence/m6_mergemania_graybox_e4_20260919 10:48:22,
    // 13/13 green). These values move ONLY with a recorded design decision,
    // never to make a red run green (SKILL: measured values go into assertions
    // as-is; thresholds are the E4 gate thresholds).
    public class MergeSimParityTests
    {
        // E4 attempt2 tuning snapshot (production contract).
        [Test]
        public void Constants_Match_E4_Attempt2_Tuning()
        {
            Assert.AreEqual(7, MergeSim.Cols, "Cols");
            Assert.AreEqual(10, MergeSim.Rows, "Rows");
            Assert.AreEqual(5, MergeSim.TopTier, "TopTier");
            Assert.AreEqual(1.0 / 60.0, MergeSim.TickSec, 1e-9, "TickSec");
            Assert.AreEqual(0.3, MergeSim.FallSec, 1e-9, "FallSec");
            Assert.AreEqual(30.0, MergeSim.RotatePeriodSec, 1e-9, "RotatePeriodSec");
            // C990 value lock (U153-D01 d-p4): 6.0 -&gt; 1.40 per the d-p3c
            // qualifying cell (pooled 0.667 in band; recorded design decision,
            // evidence = tune_sweep_calib_report_c990.json in Design/evidence).
            Assert.AreEqual(1.40, MergeSim.HeatPerMerge, 1e-9, "HeatPerMerge");
            Assert.AreEqual(100.0, MergeSim.HeatCap, 1e-9, "HeatCap");
        }
        // Scripted opening replicating the E4 stage-1 hook timeline
        // (graybox MergeHookFlow): T+0 caption, T+0.5 guided first drop,
        // T+2.0 second guided merge, T+6.0 gifted top-tier cross-family pair.
        static MergeSim RunHookOpening(int seed)
```

```
{
    var sim = new MergeSim(seed);
    sim.SeedColumn(3, new[] { 2, 3, 4 }, MergeSpecies.Produce);
    sim.Tick();
    sim.EmitExternal("caption", "Two melons collide... a CAT-MELON??");
    sim.AdvanceTo(0.5);
    sim.Drop(3, 4, MergeSpecies.Produce); // 4+4 -&gt; 5 (watermelon)
    sim.AdvanceTo(2.0);
    sim.SeedColumn(4, new[] { 1, 2, 2 }, MergeSpecies.Produce);
    sim.Drop(4, 2, MergeSpecies.Produce);  // 2+2 -&gt; 3
    sim.AdvanceTo(6.0);
    sim.SeedColumn(2, new[] { 5 }, MergeSpecies.Creature); // gifted top-tier cat
    sim.Drop(2, 5, MergeSpecies.Produce);                  // cross-family collide
    sim.AdvanceTo(10.0);
    return sim;
}
static SimEvent FirstEvent(MergeSim sim, string kind)
{
    SimEvent first = null;
    foreach (var e in sim.Events)
        if (e.Kind == kind &amp;&amp; (first == null || e.TimeSec &lt; first.TimeSec)) first = e;
    return first;
}
// E4 gate: first_merge_feedback_within_1s_of_first_drop (measured 0.82s
// after a 0.52s first op in the report; here the op moment is recovered
// from the drop landing timestamp, so the delta is the fall beat).
[Test]
public void Hook_FirstMerge_Lands_Within_1s_Of_First_Drop()
{
    var sim = new MergeSim(1);
    sim.SeedColumn(3, new[] { 2, 3, 4 }, MergeSpecies.Produce);
    sim.Tick();
    sim.AdvanceTo(0.5);
    double landSec = sim.Drop(3, 4, MergeSpecies.Produce);
    double opSec = landSec - MergeSim.FallSec;
    var firstMerge = FirstEvent(sim, "merge");
    Assert.IsNotNull(firstMerge, "hook opening must merge");
    Assert.LessOrEqual(firstMerge.TimeSec - opSec, 1.0, "first merge feedback &lt;= 1s after first drop");
    Assert.GreaterOrEqual(firstMerge.TimeSec - opSec, 0.0, "merge cannot precede its drop");
    Assert.AreEqual(landSec, firstMerge.TimeSec, 1e-6, "merge fires on landing (E4 beat ~0.82s)");
}
// E4 gate: three_merges_within_10s (attempt2 fix #1: the hybrid pair-up
// counts as a merge event, so cumulative merges in the 10s window = 3).
[Test]
public void Hook_Three_Merges_Within_10s_Window()
{
    var sim = RunHookOpening(1);
    int mergesIn10s = 0;
    foreach (var e in sim.Events)
        if (e.Kind == "merge" &amp;&amp; e.TimeSec &lt;= 10.0) mergesIn10s++;
```

```
    Assert.AreEqual(3, mergesIn10s, "E4 beat: 3 merges in the 10s hook window (0.82/2.3/6.33)");
    Assert.AreEqual(3, sim.MergeCount, "MergeCount agrees with merge events incl. hybrid");
}
// E4 gates: cat_melon_spawn_within_10s and first_addiction_signal_within_10s
// (both measured 6.33s).
[Test]
public void Hook_CatMelon_And_Laugh_Signal_Within_10s()
{
    var sim = RunHookOpening(1);
    var hybrid = FirstEvent(sim, "hybrid");
    var laugh = FirstEvent(sim, "laugh_signal");
    Assert.IsNotNull(hybrid, "hybrid spawn in hook window");
    Assert.LessOrEqual(hybrid.TimeSec, 10.0);
    Assert.Greater(hybrid.TimeSec, 6.0, "gifted pair collides after the T+6 beat");
    Assert.IsNotNull(laugh, "laugh signal in hook window");
    Assert.LessOrEqual(laugh.TimeSec, 10.0);
    Assert.AreEqual(hybrid.TimeSec, laugh.TimeSec, 1e-9, "laugh rides the hybrid beat");
    Assert.AreEqual(1, sim.HybridCount);
}
// Attempt2 semantics: hybrid pair-up emits merge + hybrid + laugh_signal,
// counts into MergeCount, and spawns a TopTier Hybrid item.
[Test]
public void Hybrid_PairUp_Emits_Merge_Hybrid_Laugh_And_Counts()
{
    var sim = new MergeSim(7);
    sim.SeedColumn(0, new[] { 5 }, MergeSpecies.Creature);
    sim.Drop(0, 5, MergeSpecies.Produce);
    Assert.IsNotNull(FirstEvent(sim, "merge"), "hybrid pair-up emits a merge event (attempt2 fix #1)");
    Assert.IsNotNull(FirstEvent(sim, "hybrid"), "hybrid beat");
    Assert.IsNotNull(FirstEvent(sim, "laugh_signal"), "laugh beat");
    Assert.AreEqual(1, sim.MergeCount, "hybrid counts as one merge");
    Assert.AreEqual(1, sim.HybridCount);
    Assert.AreEqual(5, sim.StackTopTier(0), "cat-melon sits at TopTier");
}
// Guard: cross-species pairs below TopTier never merge; same-species
// same-tier pairs do (line merge).
[Test]
public void CrossSpecies_Below_TopTier_Never_Merges_SameSpecies_Does()
{
    var sim = new MergeSim(9);
    sim.SeedColumn(0, new[] { 3 }, MergeSpecies.Creature);
    sim.Drop(0, 3, MergeSpecies.Produce);
    Assert.AreEqual(0, sim.MergeCount, "cross-species below TopTier must not pair");
    Assert.AreEqual(2, sim.ItemCount);
    sim.SeedColumn(1, new[] { 3 }, MergeSpecies.Produce);
    sim.Drop(1, 3, MergeSpecies.Produce);
    Assert.AreEqual(1, sim.MergeCount, "same-species same-tier line merge");
    Assert.AreEqual(3, sim.ItemCount, "a pair merge keeps one item per pair");
}
// Attempt2 fix #2 (rotate chain earnable): a real 90-degree transpose
```

```
// re-stacks items from different columns into one column. Fixture: single
// items in columns 0..3 with a tier ladder 3/2/1/1 -&amp;gt; after the 30s
// rotation all four land in new column 0 stacked bottom-to-top as
// [3, 2, 1, 1]; the resolve cascade pairs top-down: 1+1-&amp;gt;2 (on top of the
// seeded 2), 2+2-&amp;gt;3 (on top of the seeded 3), 3+3-&amp;gt;4 = 3 chain merges
// (chain3 + rotate_chain events). A merged item sits ON TOP of the stack,
// so a flat all-same-tier fixture would stop after one merge - the ladder
// is what makes the chain deterministic.
[Test]
public void Rotate_Transpose_Makes_Rotate_Chain_Merges_Earnable()
{
    var sim = new MergeSim(11);
    sim.SeedColumn(0, new[] { 3 }, MergeSpecies.Produce);
    sim.SeedColumn(1, new[] { 2 }, MergeSpecies.Produce);
    sim.SeedColumn(2, new[] { 1 }, MergeSpecies.Produce);
    sim.SeedColumn(3, new[] { 1 }, MergeSpecies.Produce);
    sim.AdvanceTo(30.5); // first rotation fires at the 30s cadence
    Assert.AreEqual(1, sim.RotateCount, "rotation cadence fired once");
    Assert.AreEqual(3, sim.RotateChainMerges, "post-turn cascade: 3 deterministic chain merges");
    Assert.AreEqual(1, sim.ItemCount, "four ladder items collapse to one tier-4");
    Assert.AreEqual(4, sim.StackTopTier(0), "chain ends at tier 4 in column 0");
    Assert.IsNotNull(FirstEvent(sim, "rotate_chain"), "rotate_chain event");
    Assert.IsNotNull(FirstEvent(sim, "chain3"), "chain3 event");
}
// E4 gate family: waste_heat_valve_fires. TopTier pair merges push 6.0
// heat each; at cap the valve bursts, resets to 40% and decomposes the
// most valuable stack (attempt2 bot run saw 8 bursts).
[Test]
public void WasteHeat_Valve_Bursts_At_Cap_Resets_And_Decomposes()
{
    var sim = new MergeSim(13);
    int col = 0, pairs = 0;
    while (sim.HeatBurstCount == 0 &amp;amp;&amp;amp; pairs &amp;lt; 60)
    {
        // Two drops into the SAME column so the pair merges and pushes heat.
        sim.Drop(col, 5, MergeSpecies.Produce);
        sim.Drop(col, 5, MergeSpecies.Produce);
        col = (col + 1) % MergeSim.Cols;
        pairs++;
    }
    Assert.AreEqual(1, sim.HeatBurstCount, "valve bursts exactly once in the loop");
    Assert.Less(sim.Heat, MergeSim.HeatCap, "heat resets below cap after burst");
    Assert.IsNotNull(FirstEvent(sim, "heat_burst"), "heat_burst event");
    Assert.IsNotNull(FirstEvent(sim, "decompose"), "decompose event (waste-heat valve split)");
}
// Determinism contract (fixed seed): same seed + same scripted play -&amp;gt;
// identical counters and identical (time, kind) event sequence.
[Test]
public void Same_Seed_Same_Play_Is_Identical()
{
```

```
            string Play(int seed)
            {
                var sim = new MergeSim(seed);
                sim.SeedColumn(3, new[] { 2, 3, 4 }, MergeSpecies.Produce);
                sim.Tick();
                sim.AdvanceTo(0.5);
                sim.Drop(3, 4, MergeSpecies.Produce);
                sim.AdvanceTo(2.0);
                sim.SeedColumn(4, new[] { 1, 2, 2 }, MergeSpecies.Produce);
                sim.Drop(4, 2, MergeSpecies.Produce);
                var sb = new StringBuilder();
                for (int i = 0; i &lt; 20; i++)
                {
                    int tier = sim.NextSpawnTier(1, MergeSim.TopTier);
                    sim.Drop(i % MergeSim.Cols, tier, MergeSpecies.Produce);
                    sb.Append(tier).Append(';');
                }
                sb.Append("D=").Append(sim.DropCount).Append("M=").Append(sim.MergeCount);
                foreach (var e in sim.Events) sb.Append('|').Append(e.TimeSec.ToString("F4")).Append(',').Append(e.Kind)
                return sb.ToString();
            }
            Assert.AreEqual(Play(42), Play(42), "fixed seed play must be identical");
        }
        // E4 stage-3 gates: drop_resolve_under_3ms (measured 0.0136ms) and
        // tick_under_0_05ms (measured 0.00002ms) on a dense board.
        [Test]
        public void Perf_Dense_Drop_Under_3ms_Tick_Under_0_05ms()
        {
            var sim = new MergeSim(17);
            // Dense board: every column 8 deep, tops tier 2 so each measured drop
            // resolves through a merge (worst realistic resolve path).
            for (int c = 0; c &lt; MergeSim.Cols; c++)
                sim.SeedColumn(c, new[] { 1, 1, 2, 2, 3, 3, 4, 2 }, MergeSpecies.Produce);
            var sw = Stopwatch.StartNew();
            const int measuredDrops = 23; // E4 dense_drops_measured=23
            for (int i = 0; i &lt; measuredDrops; i++)
                sim.Drop(i % MergeSim.Cols, 2, MergeSpecies.Produce);
            sw.Stop();
            double perDropMs = sw.Elapsed.TotalMilliseconds / measuredDrops;
            Assert.Less(perDropMs, 3.0, "E4 gate: drop resolve under 3ms, measured " + perDropMs.ToString("F4"));
            var sim2 = new MergeSim(17);
            var sw2 = Stopwatch.StartNew();
            const int ticks = 60000; // ~1000 virtual seconds incl. ~33 rotations
            for (int i = 0; i &lt; ticks; i++) sim2.Tick();
            sw2.Stop();
            double perTickMs = sw2.Elapsed.TotalMilliseconds / ticks;
            Assert.Less(perTickMs, 0.05, "E4 gate: raw tick under 0.05ms, measured " + perTickMs.ToString("F5"));
        }
    }
}
```